

CME 295: Transformers & Large Language Models

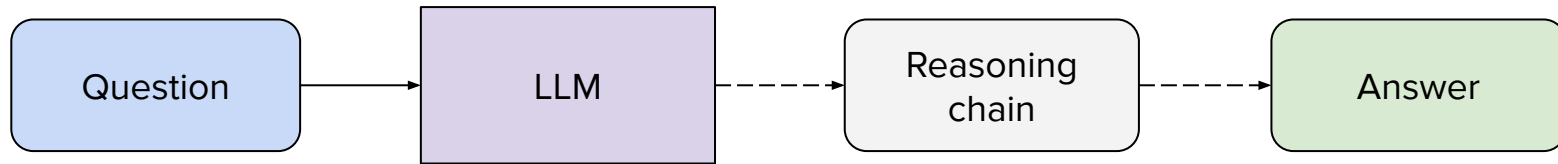


Afshine Amidi & Shervine Amidi



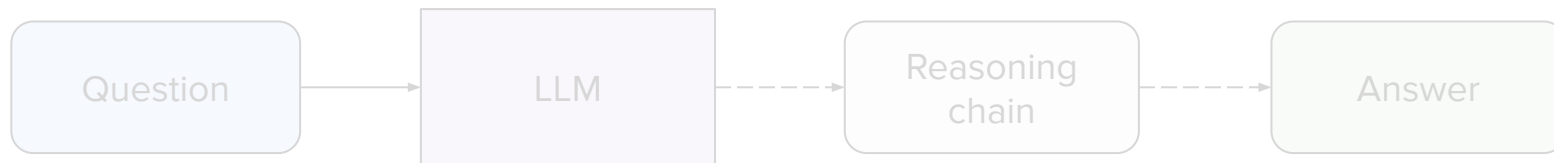
Recap of last episode...

Reasoning models

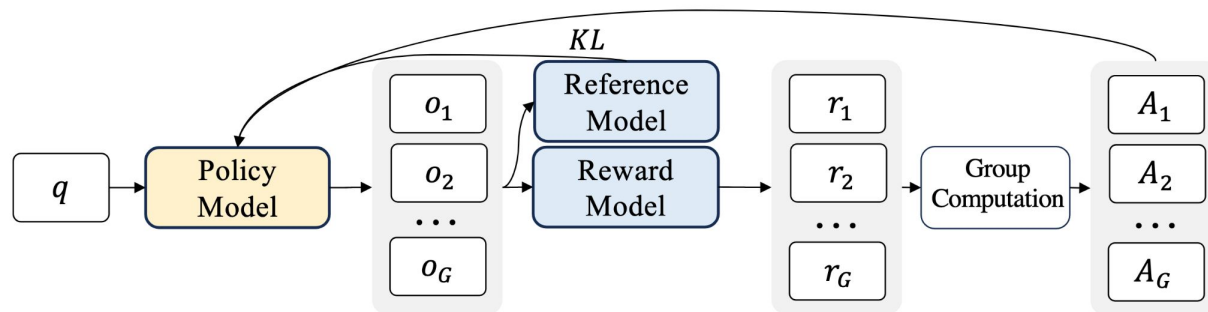


Recap of last episode...

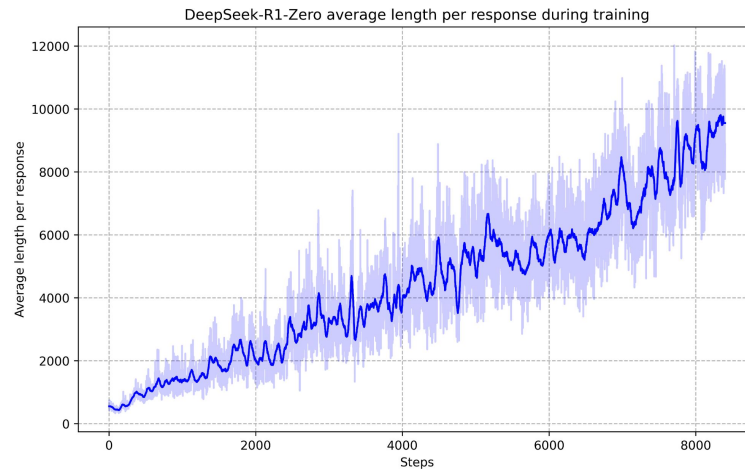
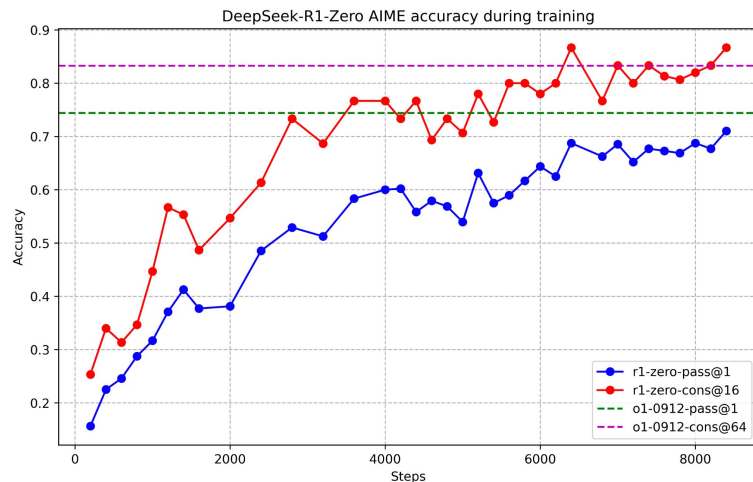
Reasoning models



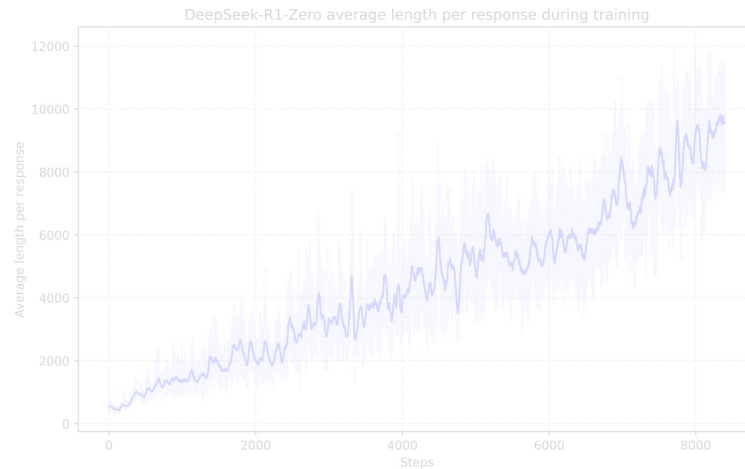
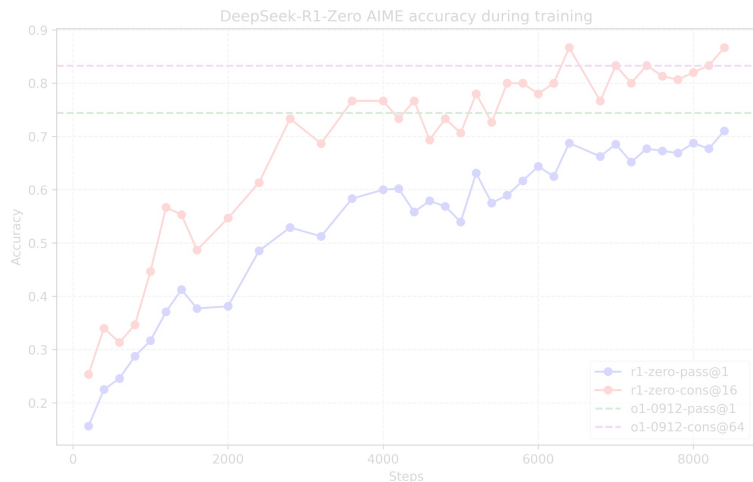
GRPO



Recap of last episode...

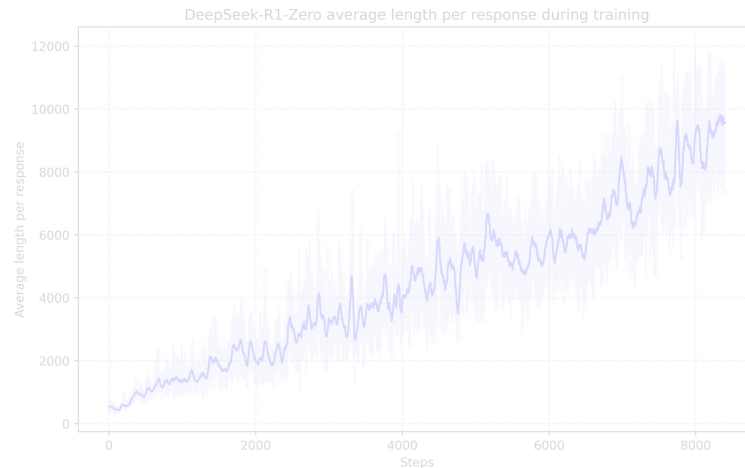
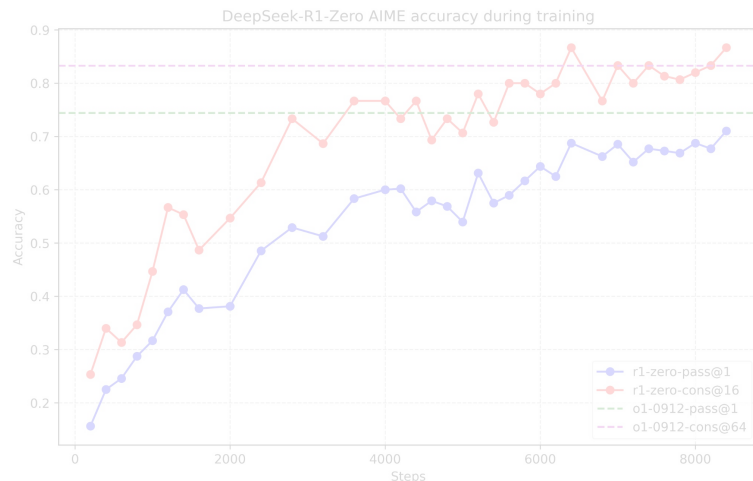


Recap of last episode...



Original $\frac{1}{G} \sum_{i=1}^G \frac{1}{|\mathbf{o}_i|} \sum_{t=1}^{|\mathbf{o}_i|}$

Recap of last episode...



Original $\frac{1}{G} \sum_{i=1}^G \frac{1}{|\mathbf{o}_i|} \sum_{t=1}^{|\mathbf{o}_i|}$

DAPO $\frac{1}{\sum_{i=1}^G |\mathbf{o}_i|} \sum_{i=1}^G \sum_{t=1}^{|\mathbf{o}_i|}$

Dr. GRPO $\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|\mathbf{o}_i|}$

"Understanding R1-Zero-Like Training: A Critical Perspective", Liu et al., 2025.

"DAPO: An Open-Source LLM Reinforcement Learning System at Scale", Yu et al., 2025.

Recap of last episode...

Strengths.

- Great at imitation or idea generation
- Amazing at generating or debugging code

Weaknesses.

- Limited reasoning
- Knowledge is static
- Cannot perform actions
- Hard to evaluate

Recap of last episode...

Strengths.

- Great at imitation or idea generation
- Amazing at generating or debugging code

Weaknesses.

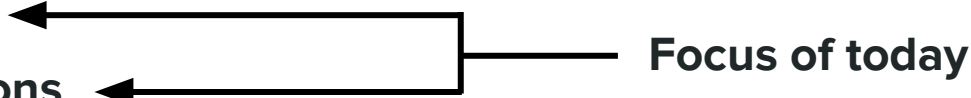
- Limited reasoning
 - Knowledge is static
 - Cannot perform actions
 - Hard to evaluate
- ← Focus of last lecture

Recap of last episode...

Strengths.

- Great at imitation or idea generation
- Amazing at generating or debugging code

Weaknesses.

- Limited reasoning
 - **Knowledge is static**
 - **Cannot perform actions**
 - Hard to evaluate
- 
- The diagram consists of a horizontal line on the right labeled 'Focus of today'. From the left end of this line, two parallel arrows point leftwards. The top arrow points to the text 'Knowledge is static' and the bottom arrow points to the text 'Cannot perform actions'.



Transformers & Large Language Models

RAG

Tool calling

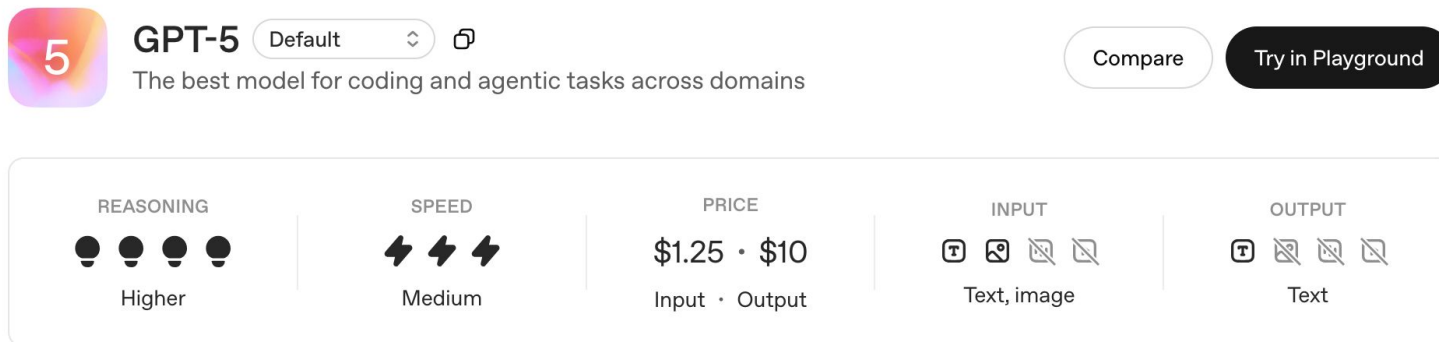
Agents

Motivation

- **Knowledge** of LLM **constrained** to pretraining data

Motivation

- **Knowledge of LLM constrained** to pretraining data

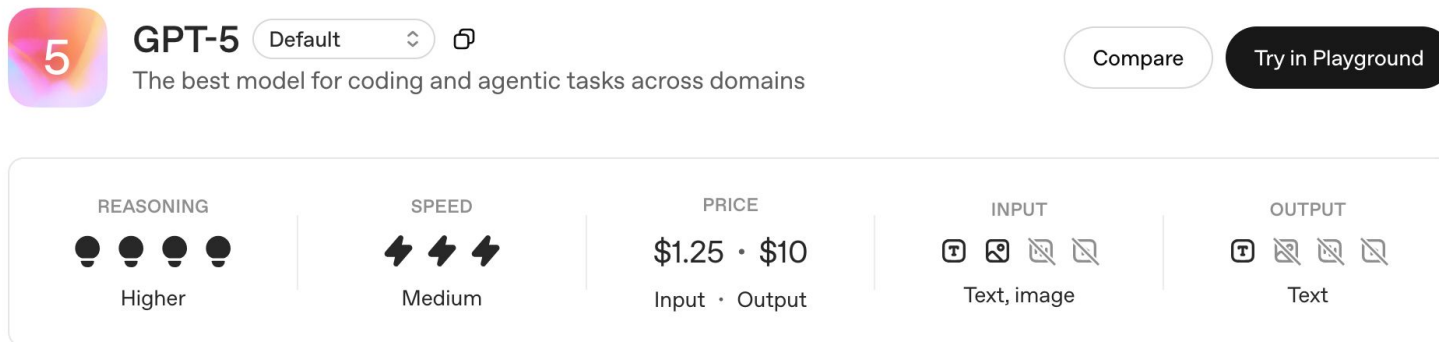


GPT-5 is our flagship model for coding, reasoning, and agentic tasks across domains. Learn more in our [GPT-5 usage guide](#).

- ✦ 400,000 context window
- ↪ 128,000 max output tokens
- 📅 Sep 30, 2024 knowledge cutoff
- 💡 Reasoning token support

Motivation

- **Knowledge of LLM constrained** to pretraining data



The screenshot shows the OpenAI GPT-5 model card. At the top left is a logo with the number '5'. Next to it is the text 'GPT-5' followed by a 'Default' dropdown menu and a share icon. To the right are two buttons: 'Compare' and 'Try in Playground'. Below this is a performance comparison table with five columns: REASONING, SPEED, PRICE, INPUT, and OUTPUT. Each column has icons and a descriptive text.

REASONING	SPEED	PRICE	INPUT	OUTPUT
Higher	Medium	\$1.25 · \$10	Text, image	Text

GPT-5 is our flagship model for coding, reasoning, and agentic tasks across domains. Learn more in our [GPT-5 usage guide](#).

- ✦ 400,000 context window
- ↪ 128,000 max output tokens
- 📅 Sep 30, 2024 knowledge cutoff
- 💡 Reasoning token support

Motivation

- Knowledge of LLM **constrained** to pretraining data
- **Limited** context size

Motivation

- Knowledge of LLM constrained to pretraining data
- **Limited** context size

The screenshot shows the OpenAI GPT-5 model card. At the top left is a colorful square icon with the number '5'. To its right is the text 'GPT-5' followed by a 'Default' dropdown menu and a share icon. Below this is the tagline 'The best model for coding and agentic tasks across domains'. To the right are two buttons: 'Compare' and 'Try in Playground'. Below these elements is a horizontal bar with five categories: REASONING (four circles, 'Higher'), SPEED (three lightning bolts, 'Medium'), PRICE ('\$1.25 · \$10', 'Input · Output'), INPUT (icons for text, image, and two crossed-out image icons, 'Text, image'), and OUTPUT (text icon and three crossed-out image icons, 'Text').

GPT-5 is our flagship model for coding, reasoning, and agentic tasks across domains. Learn more in our [GPT-5 usage guide](#).

✦ 400,000 context window

↪ 128,000 max output tokens

📅 Sep 30, 2024 knowledge cutoff

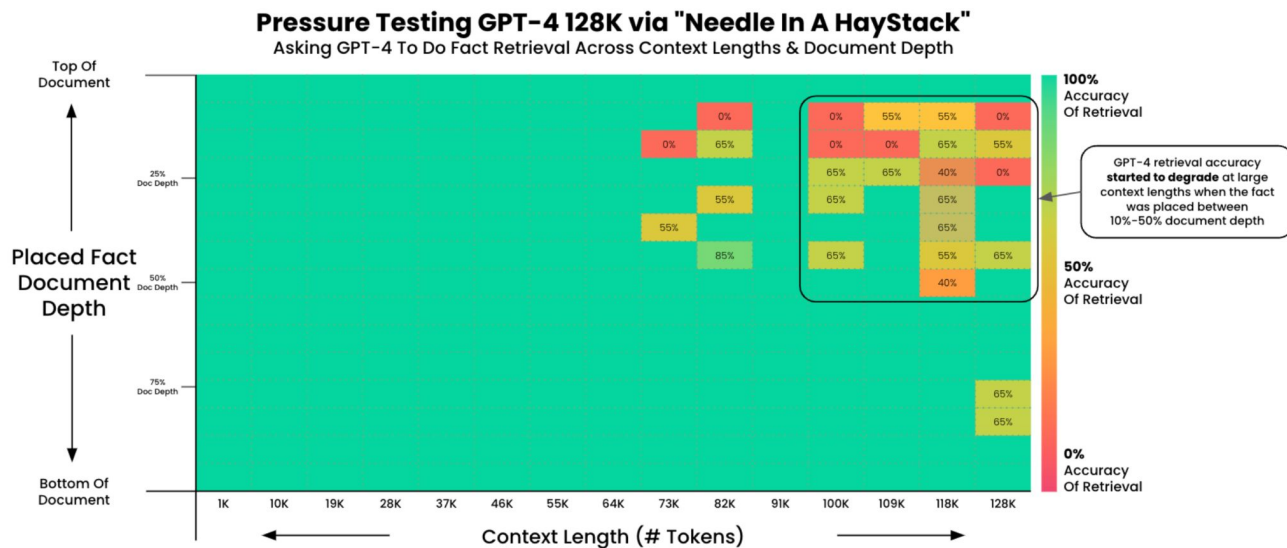
💡 Reasoning token support

Motivation

- Knowledge of LLM **constrained** to pretraining data
- **Limited** context size
- LLM gets **distracted** by **useless** information

Motivation

- Knowledge of LLM constrained to pretraining data
- Limited context size
- LLM gets **distracted** by **useless** information



Motivation

- **Knowledge** of LLM **constrained** to pretraining data
- **Limited** context size
- LLM gets **distracted** by **useless** information
- **Pricing** is **per** input/output **token**

Motivation

- Knowledge of LLM **constrained** to pretraining data
- **Limited** context size
- LLM gets **distracted** by **useless** information
- **Pricing** is **per input/output token**

 **GPT-5** Default 🔄 📄

The best model for coding and agentic tasks across domains

Compare Try in Playground

REASONING

● ● ● ●

Higher

SPEED

⚡ ⚡ ⚡

Medium

PRICE

\$1.25 · \$10

Input · Output

INPUT

T 📷 🚫 🚫

Text, image

OUTPUT

T 🚫 🚫 🚫

Text

RAG = **R**etrieval-**A**ugmented **G**eneration

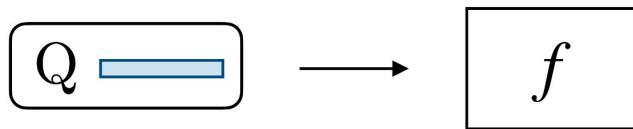
RAG = Retrieval-Augmented Generation

Idea. Augment prompt with **relevant** pieces of information.

Overview

RAG = Retrieval-Augmented Generation

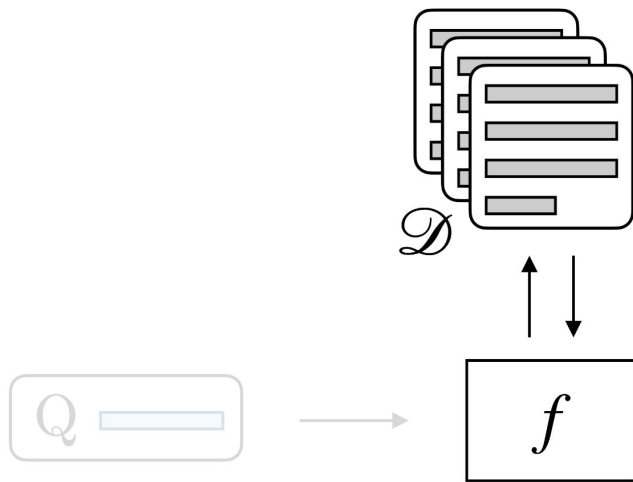
Idea. Augment prompt with **relevant** pieces of information.



Overview

RAG = Retrieval-Augmented Generation

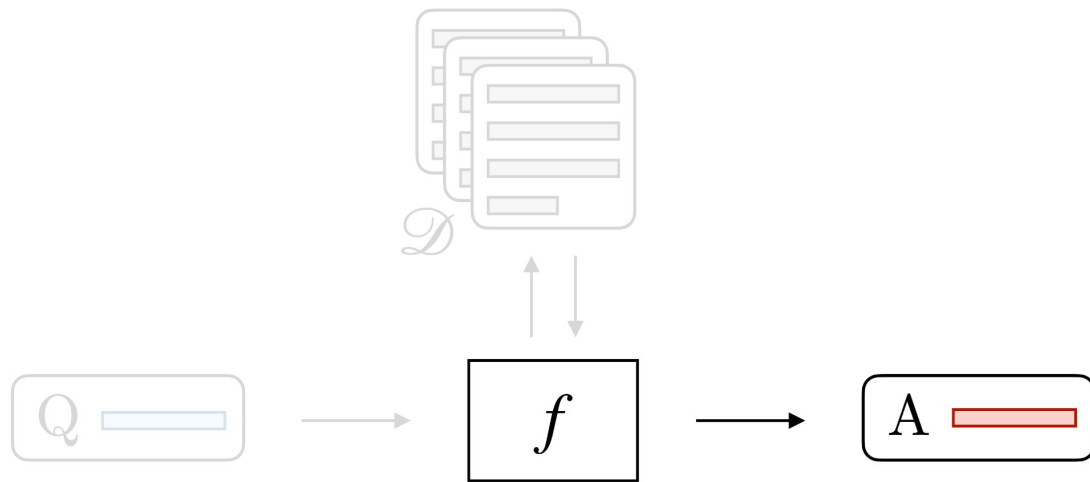
Idea. Augment prompt with **relevant** pieces of information.



Overview

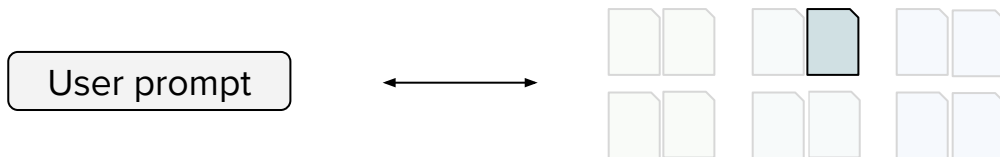
RAG = Retrieval-Augmented Generation

Idea. Augment prompt with **relevant** pieces of information.



Retrieve, Generate, Augment

1. **Retrieve** relevant document via similarity operation across the knowledge base

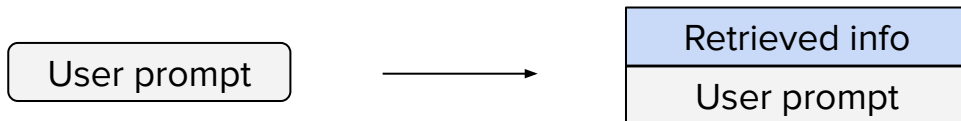


Retrieve, Generate, Augment

1. **Retrieve** relevant document via similarity operation across the knowledge base



2. **Augment** prompt with retrieved information



Retrieve, Generate, Augment

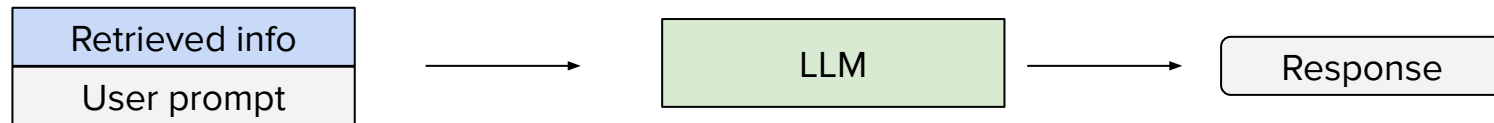
1. **Retrieve** relevant document via similarity operation across the knowledge base



2. **Augment** prompt with retrieved information



3. **Generate** response



Retrieve, Generate, Augment

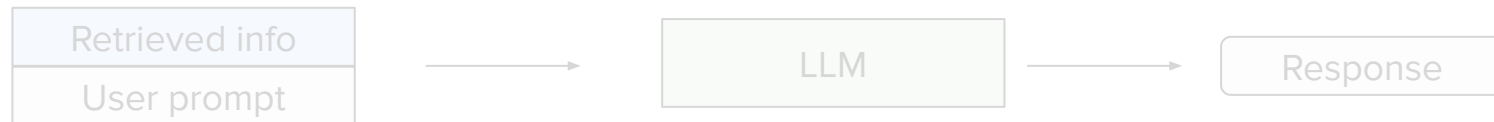
1. **Retrieve** relevant document via similarity operation across the knowledge base



2. **Augment** prompt with retrieved information

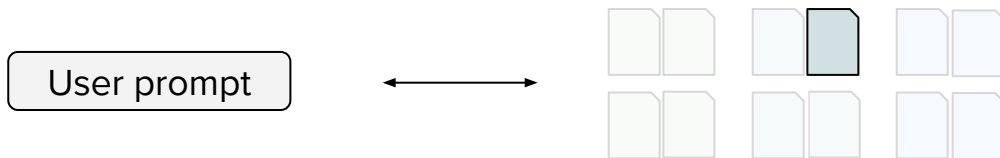


3. **Generate** response



Focus on retrieval stage

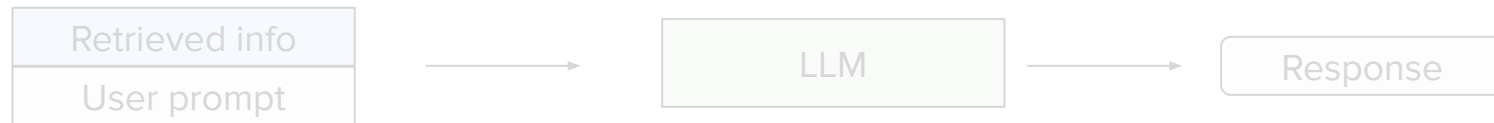
1. **R**etrieve relevant document via similarity operation across the knowledge base



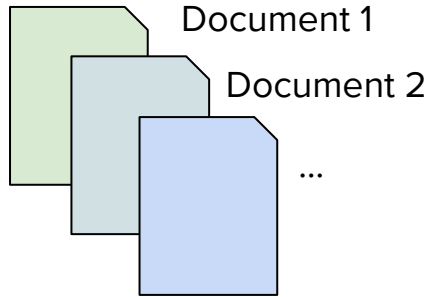
2. **A**ugment prompt with retrieved information



3. **G**enerate response

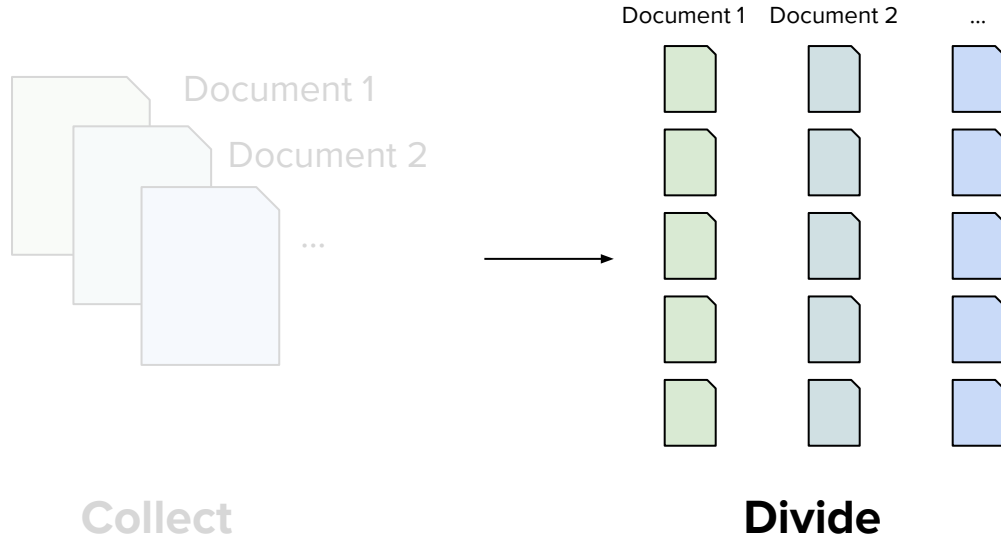


Prerequisite: Create knowledge base

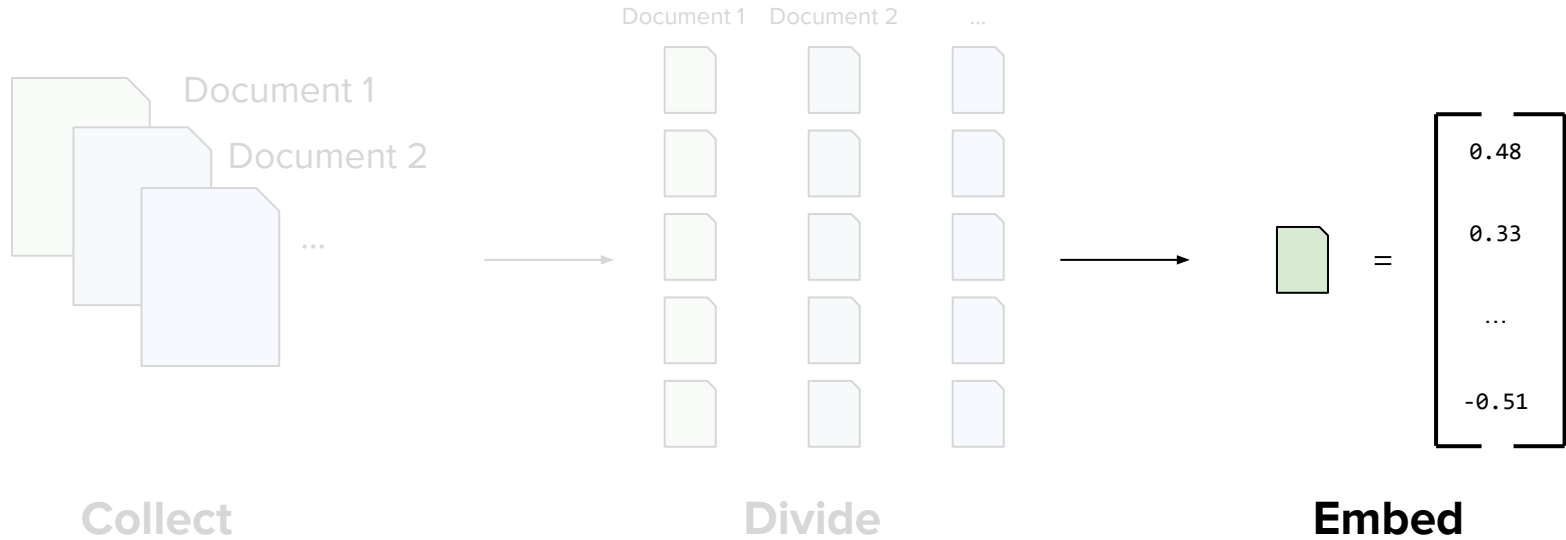


Collect

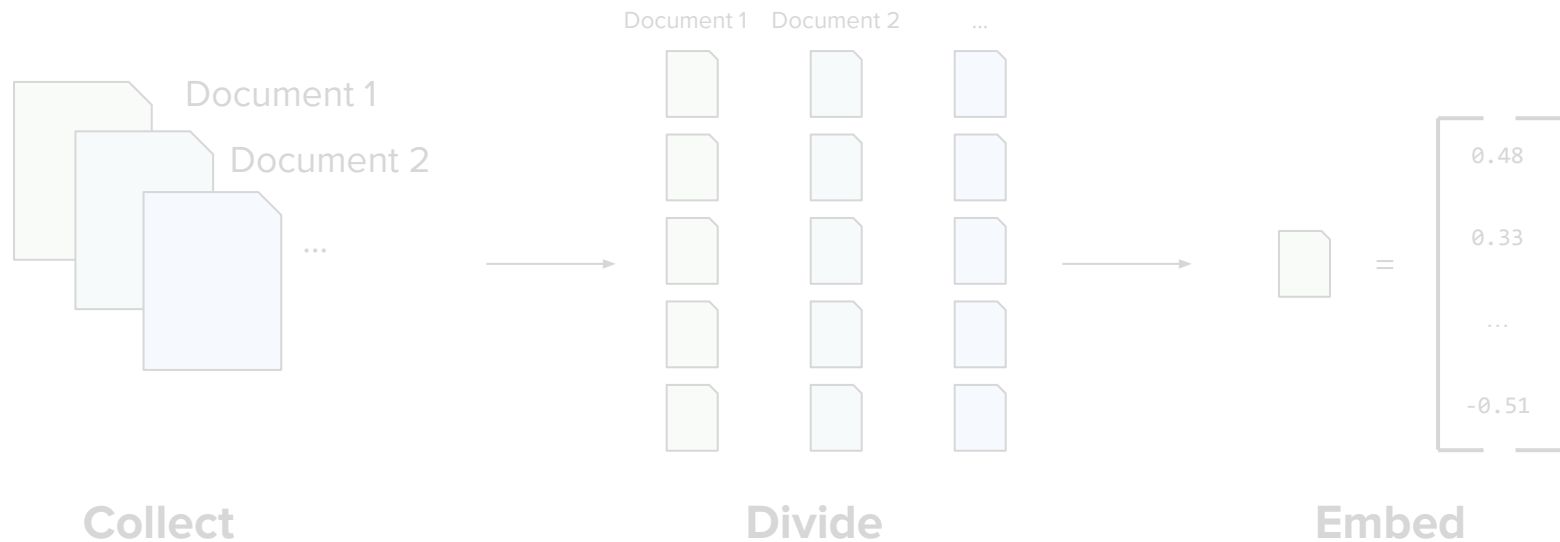
Prerequisite: Create knowledge base



Prerequisite: Create knowledge base



Prerequisite: Create knowledge base



Hyperparameters. Embedding size, chunk size, overlap between chunks

Retrieval overview



Step 1 – Candidate retrieval: Select potentially-relevant candidates

- Maximize recall
- Semantic embeddings and optionally keyword-based methods

Retrieval overview



Step 1 – Candidate retrieval: Select potentially-relevant candidates

- Maximize recall
- Semantic embeddings and optionally keyword-based methods



Step 2 – Ranking: Give final relevance score

- Maximize precision
- Re-ranking on smaller set of candidates

Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

Step 1: candidate retrieval

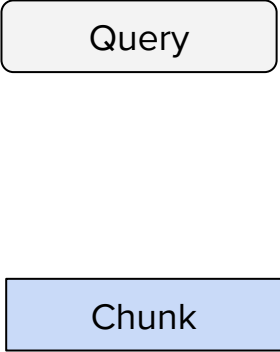
Objective. Select potentially-relevant candidates via search across knowledge base

Method 1. Semantic search using **embeddings-based similarity**

Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

Method 1. Semantic search using **embeddings-based similarity**



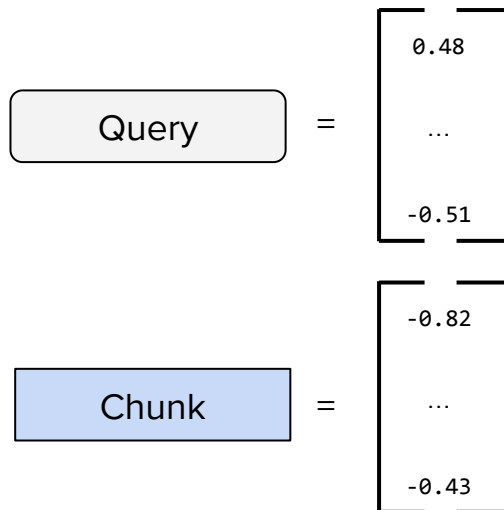
Query

Chunk

Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

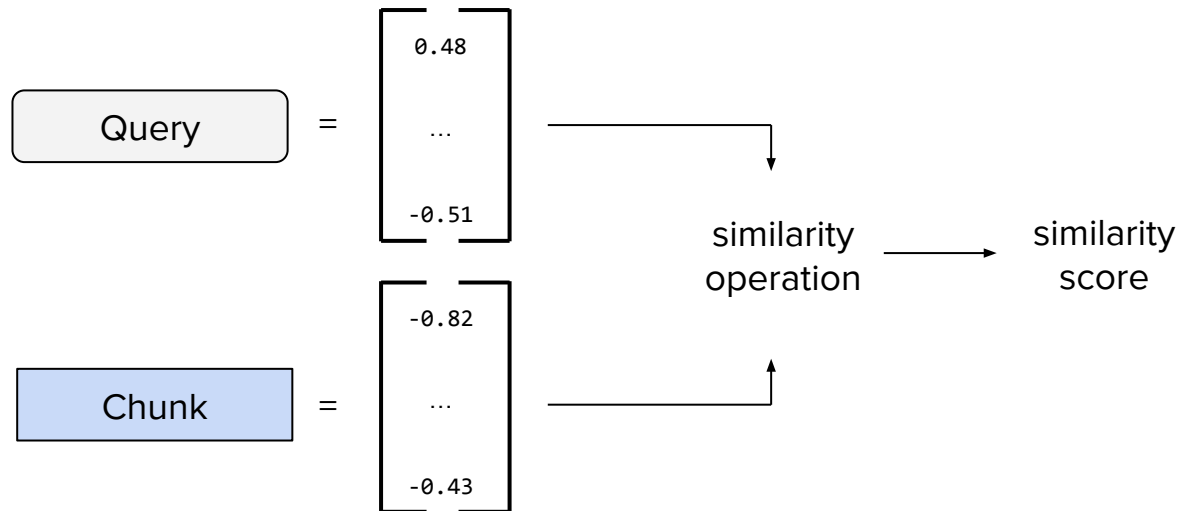
Method 1. Semantic search using **embeddings-based similarity**



Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

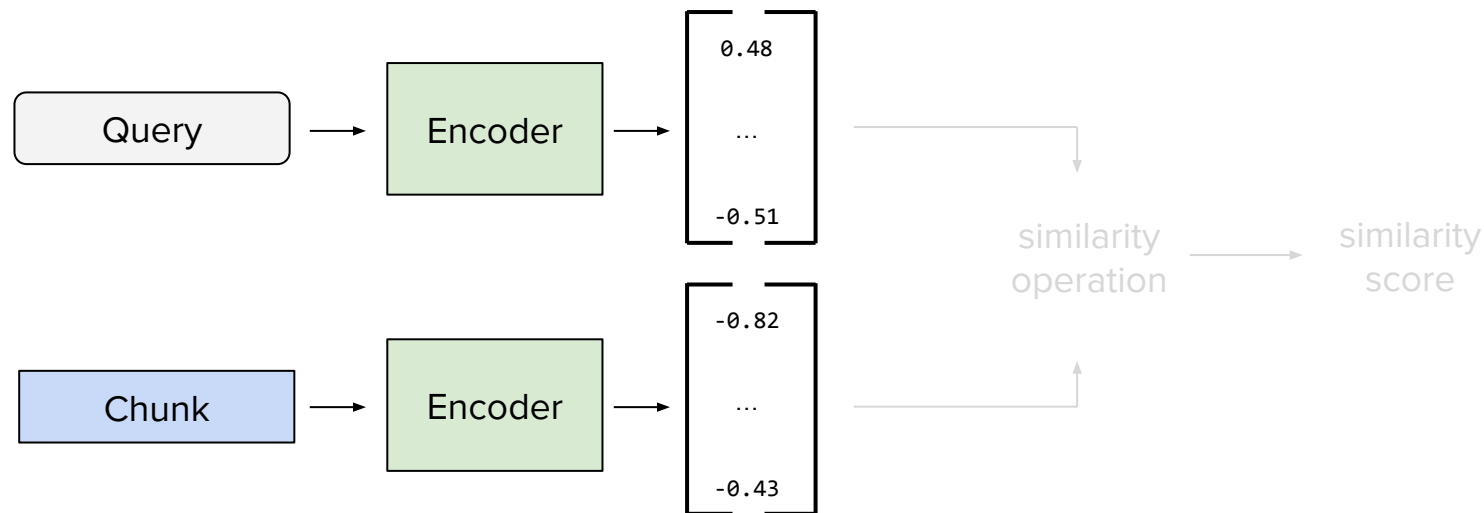
Method 1. Semantic search using **embeddings-based similarity**



Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

Method 1. Semantic search using **embeddings-based similarity**

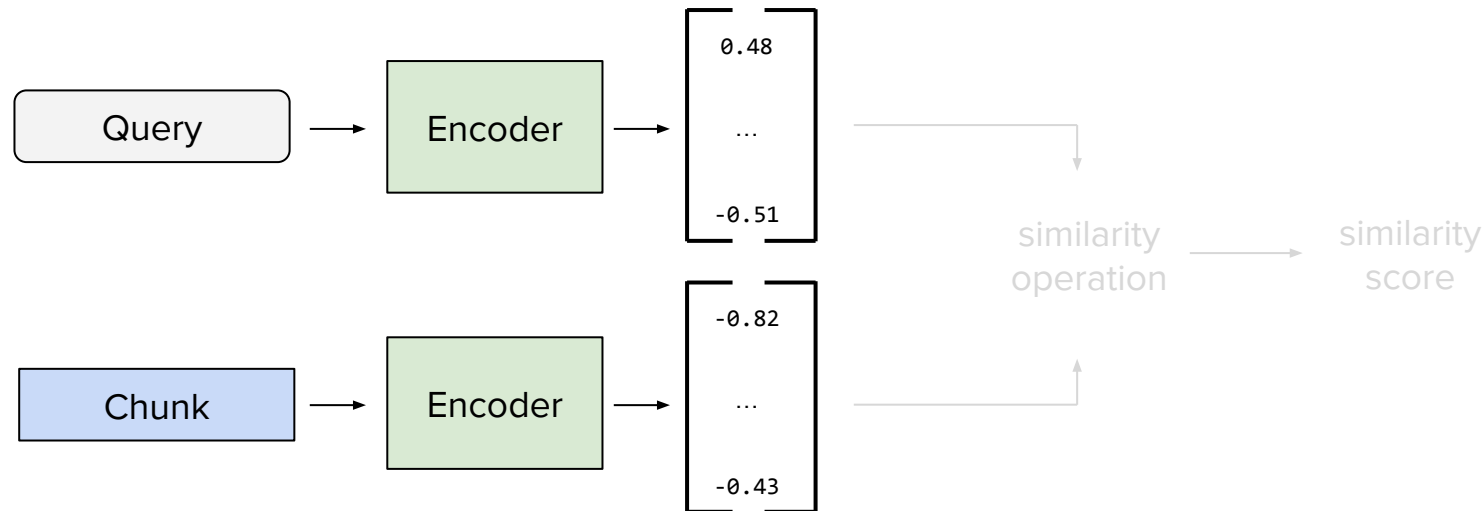


Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

Method 1. Semantic search using **embeddings-based similarity**

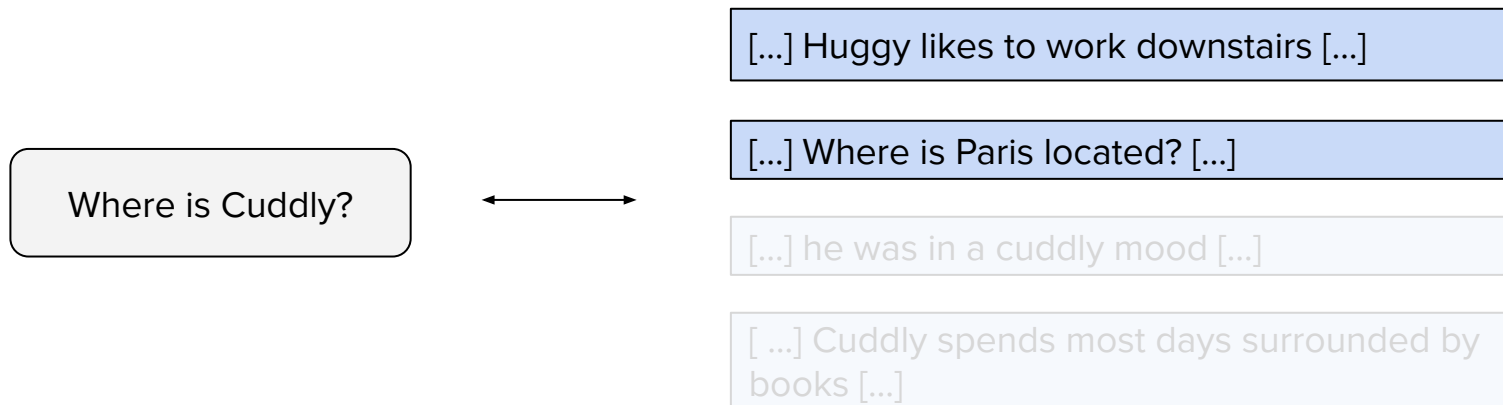
"bi-encoder"



Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

Method 1. Semantic search using **embeddings-based similarity**



Step 1: candidate retrieval

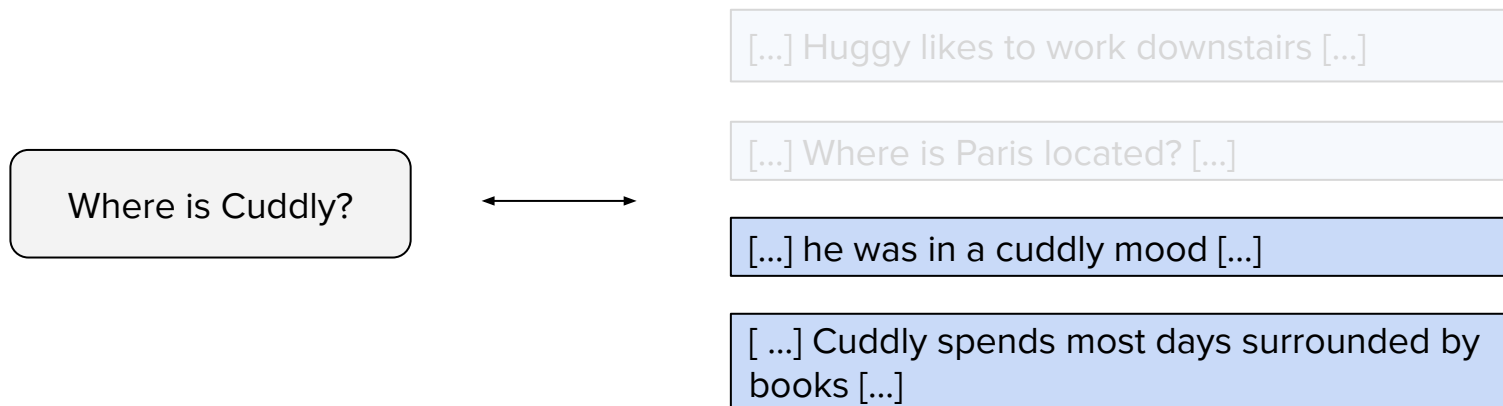
Objective. Select potentially-relevant candidates via search across knowledge base

Method 2. Search based on keywords matching via **BM25**

Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

Method 2. Search based on keywords matching via **BM25**



Step 1: candidate retrieval

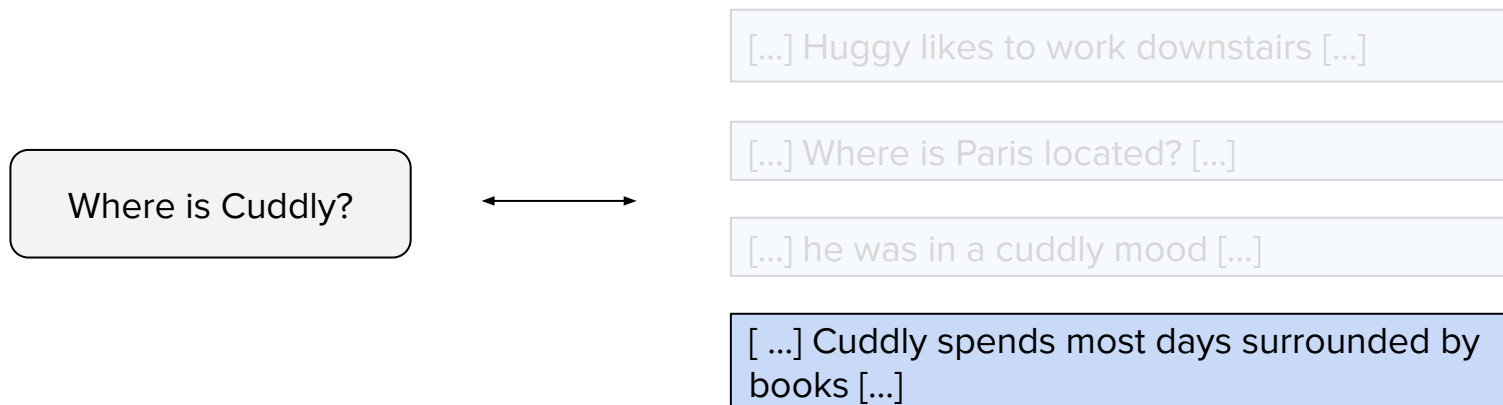
Objective. Select potentially-relevant candidates via search across knowledge base

Method 3. Search based on **hybrid combination** of semantics and BM25

Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

Method 3. Search based on **hybrid combination** of semantics and BM25



Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings

Extensions that can help with initial retrieval

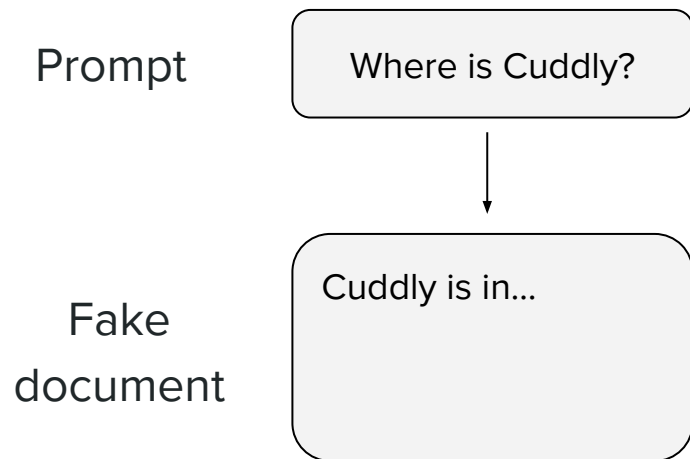
- Mitigate **discrepancy** in nature of embeddings

Prompt

Where is Cuddly?

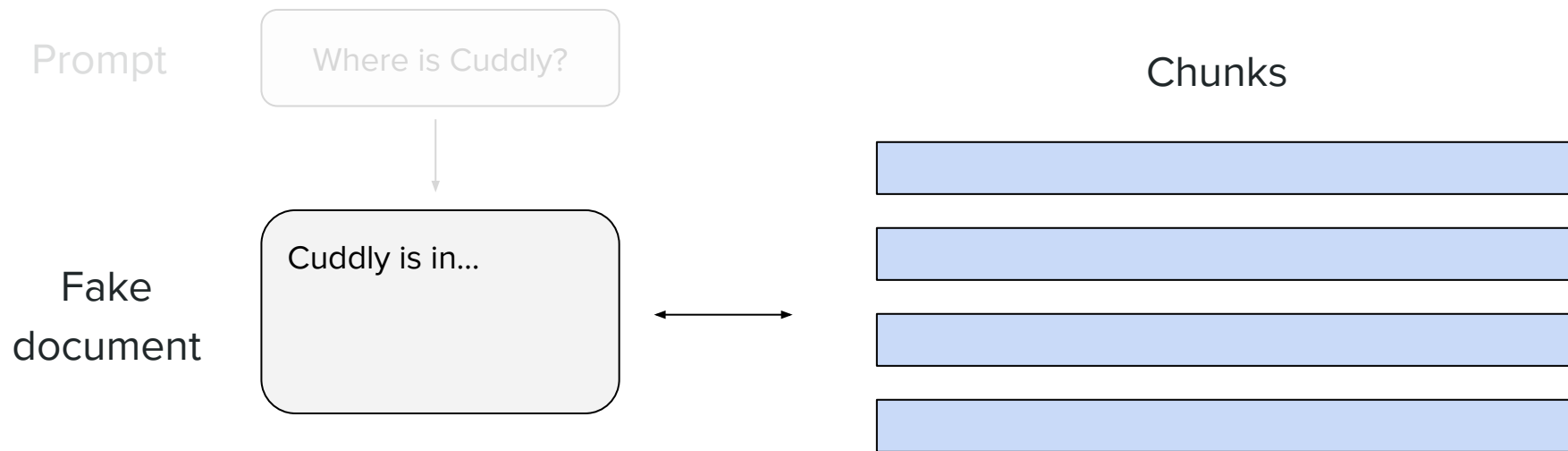
Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings



Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings



Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks

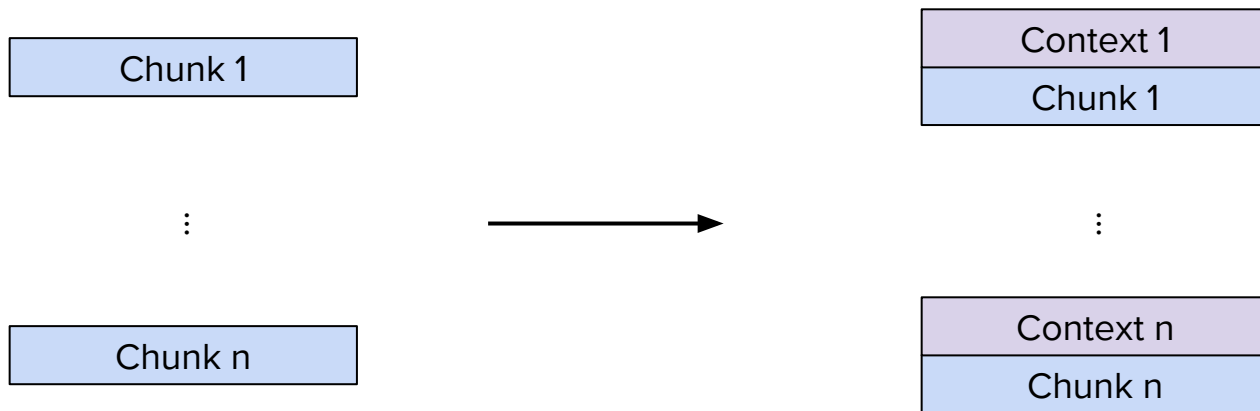
Chunk 1

⋮

Chunk n

Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks



Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks

```
<document>  
{WHOLE_DOCUMENT}  
</document>
```

Here is the chunk we want to situate within the whole document:

```
{CHUNK_CONTENT}
```

Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context and nothing else.

Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks

Prompt caching



```
<document>
{WHOLE_DOCUMENT}
</document>
```

```
Here is the chunk we want to situate within the whole document:
{CHUNK_CONTENT}
```

Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context and nothing else.

Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks

Pricing

 Copy page

Text tokens

Prices per 1M tokens.

MODEL	INPUT	CACHED INPUT	OUTPUT
gpt-5	\$1.25	\$0.125	\$10.00
gpt-5-mini	\$0.25	\$0.025	\$2.00
gpt-5-nano	\$0.05	\$0.005	\$0.40
gpt-5-chat-latest	\$1.25	\$0.125	\$10.00
gpt-5-codex	\$1.25	\$0.125	\$10.00

Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks

Pricing

 Copy page

Text tokens

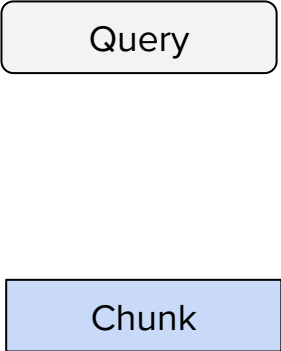
Prices per 1M tokens.

Prices per 1M tokens.

		Batch	Flex	Standard	Priority
MODEL	INPUT	CACHED INPUT			OUTPUT
gpt-5	\$1.25	\$0.125			\$10.00
gpt-5-mini	\$0.25	\$0.025			\$2.00
gpt-5-nano	\$0.05	\$0.005			\$0.40
gpt-5-chat-latest	\$1.25	\$0.125			\$10.00
gpt-5-codex	\$1.25	\$0.125			\$10.00

Step 2: ranking

Objective. Provide final relevance score using more sophisticated (re-)ranker

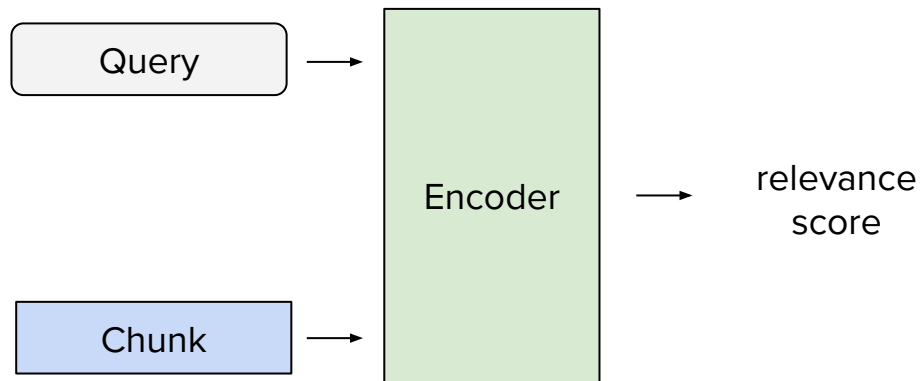


Query

Chunk

Step 2: ranking

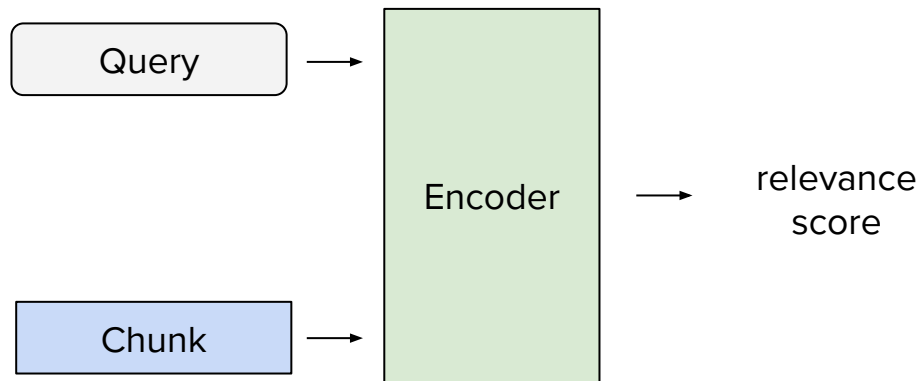
Objective. Provide final relevance score using more sophisticated (re-)ranker



Step 2: ranking

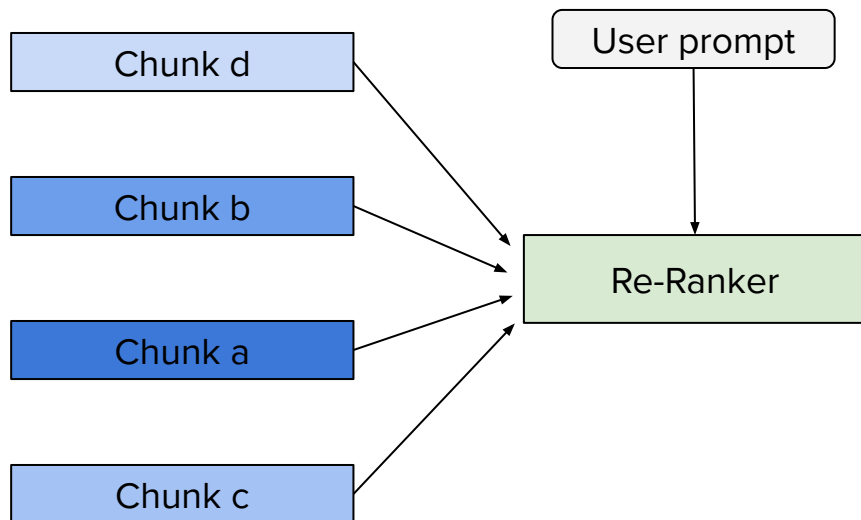
Objective. Provide final relevance score using more sophisticated (re-)ranker

"cross-encoder"



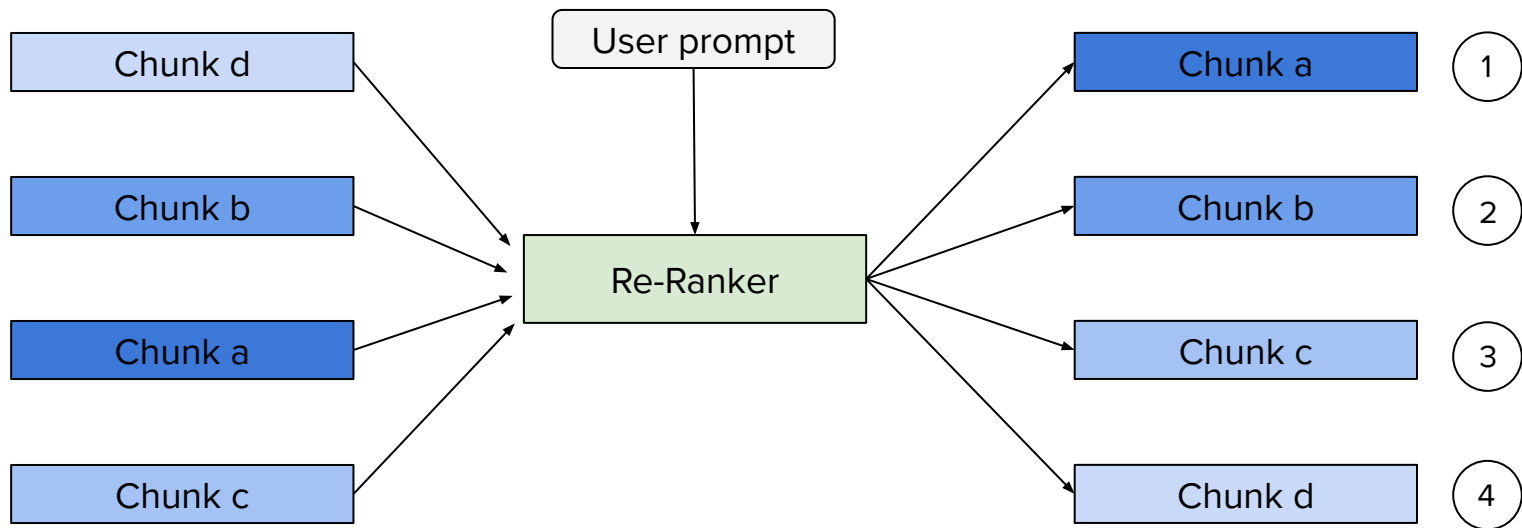
Step 2: ranking

Objective. Provide final relevance score using more sophisticated (re-)ranker



Step 2: ranking

Objective. Provide final relevance score using more sophisticated (re-)ranker

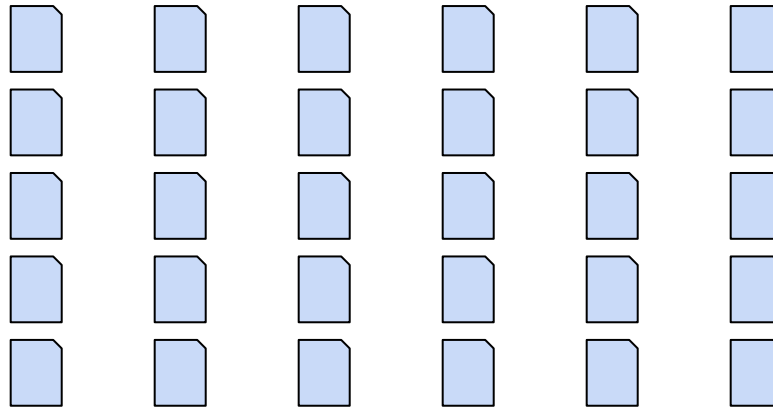


Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Quantify performance of retrieval

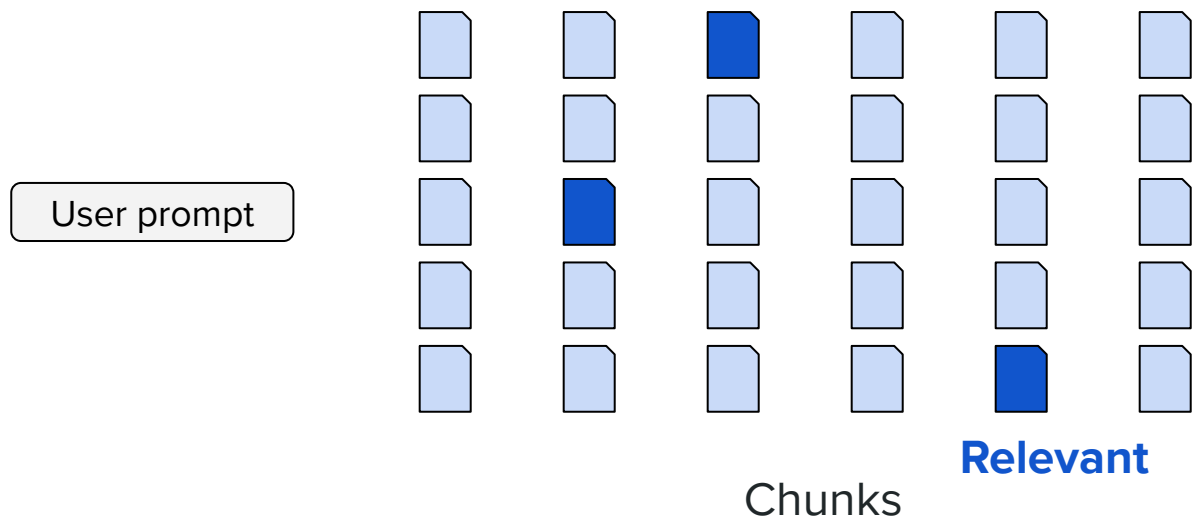
Setup. Evaluate if **retrieved chunks** are **relevant**



Chunks

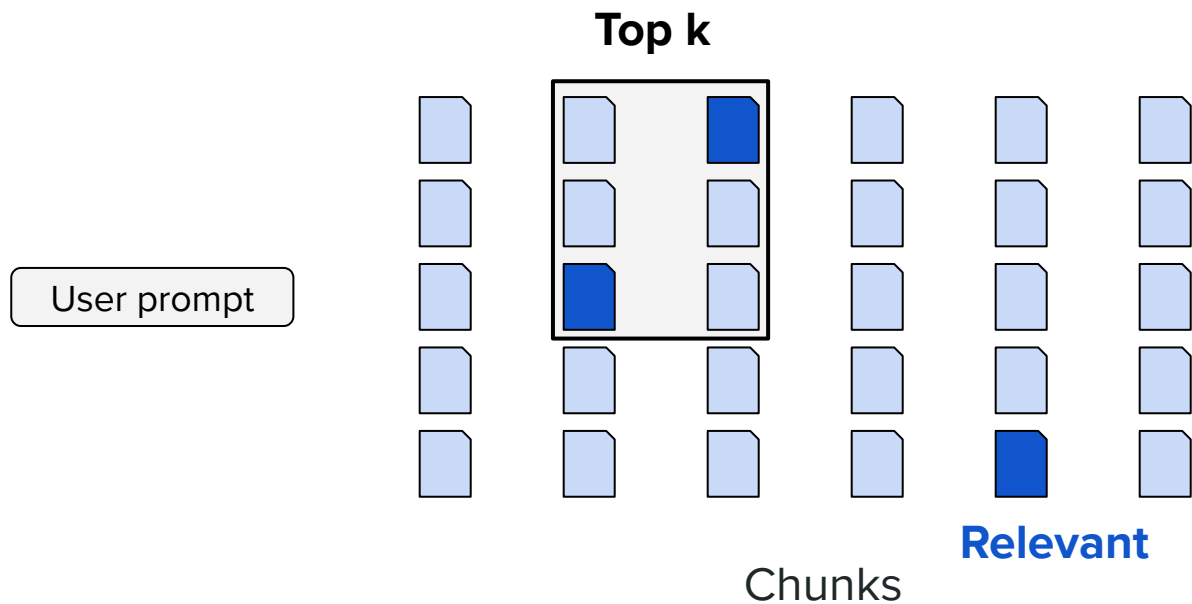
Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**



Quantify performance of retrieval

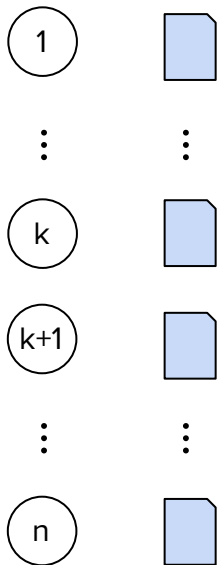
Setup. Evaluate if **retrieved chunks** are **relevant**



Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

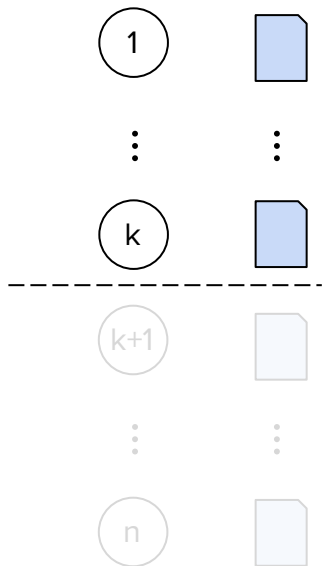
Ranking



Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

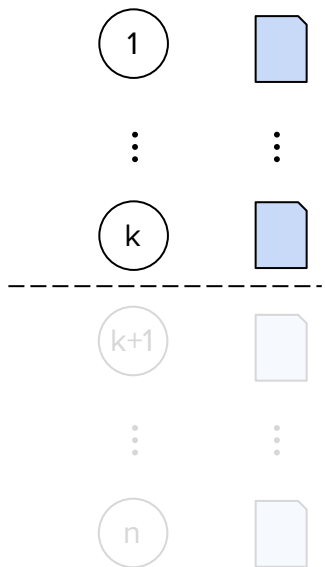
Ranking



Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Ranking

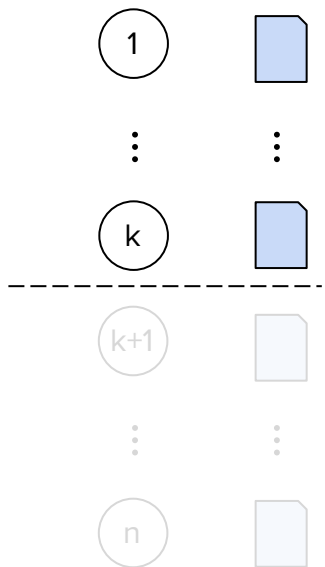


- **N**ormalized **D**iscounted **C**umulative **G**ain at **k** (**NDCG@k**)

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Ranking



- **N**ormalized **D**iscounted **C**umulative **G**ain at **k** (**NDCG@k**)

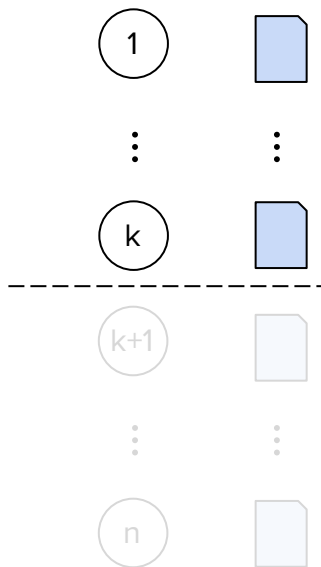
$$\text{DCG@}k = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i + 1)}$$

with $\text{rel}_i \in \{0, 1\}$

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Ranking



- **N**ormalized **D**iscounted **C**umulative **G**ain at **k** (**NDCG@k**)

$$\text{DCG}@k = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+1)}$$

with $\text{rel}_i \in \{0, 1\}$

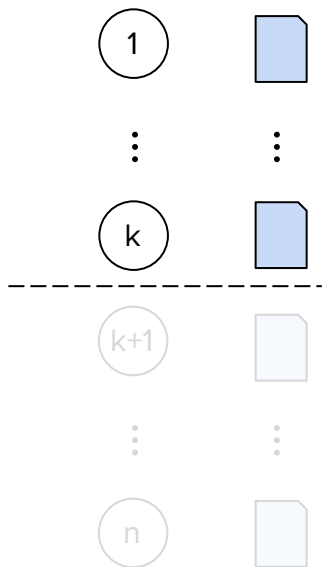
$$\text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}$$

↑
 $\text{DCG}@k$ if ranking was perfect

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Ranking



- Normalized Discounted Cumulative Gain at k (**NDCG@k**)
- Reciprocal Rank at k (**RR@k**)

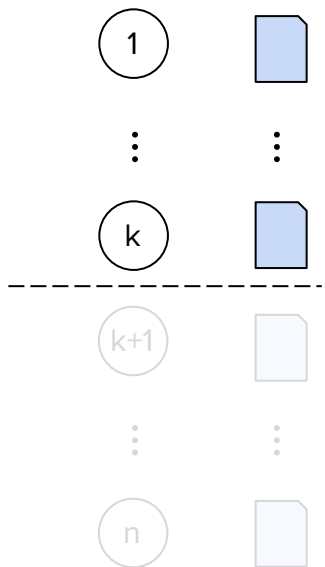
$$RR = \frac{1}{\text{rank}}$$

rank of the first relevant chunk

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Ranking

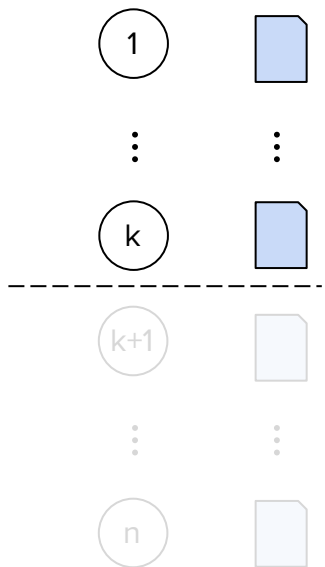


- Normalized **D**iscounted **C**umulative **G**ain at **k** (**NDCG@k**)
- Reciprocal **R**ank at **k** (**RR@k**)
- **Recall at k**

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Ranking



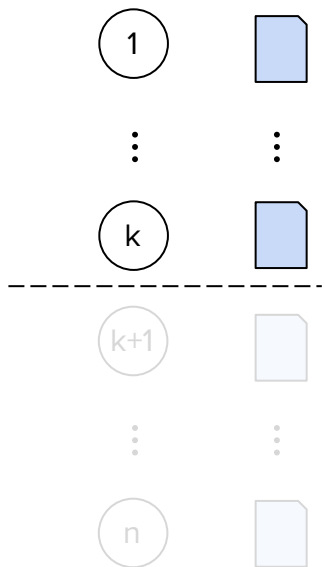
- Normalized **D**iscounted **C**umulative **G**ain at **k** (**NDCG@k**)
- Reciprocal **R**ank at **k** (**RR@k**)
- **Recall at k**

$$\text{Recall@}k = \frac{|\text{relevant in top } k|}{|\text{relevant}|}$$

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Ranking

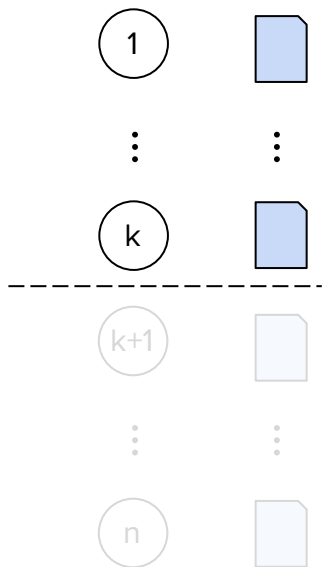


- Normalized Discounted Cumulative Gain at k (**NDCG@k**)
- Reciprocal Rank at k (**RR@k**)
- Recall at k
- **Precision at k**

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Ranking



- Normalized Discounted Cumulative Gain at k (**NDCG@k**)
- Reciprocal Rank at k (**RR@k**)
- Recall at k
- **Precision at k**

$$\text{Precision@}k = \frac{|\text{relevant in top } k|}{k}$$



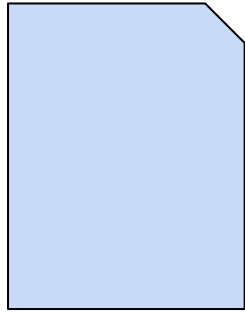
Transformers & Large Language Models

RAG

Tool calling

Agents

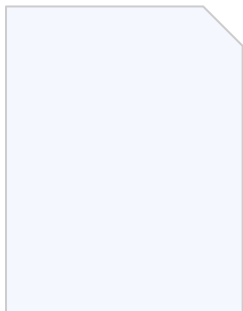
Motivation



Unstructured

RAG

Motivation



Unstructured

RAG

ID	Field	...
123	Obs	...
...	...	...

Structured

Idea

ID	Field	...
123	Obs	...
...	...	...



Function

```
def get_data(  
    id, field, ...  
):  
    # Logic.  
    ...  
    return result
```

"Tool calling [...] allows autonomous systems to complete complex tasks by dynamically accessing and [may act] upon external resources."

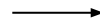
Definition

"Tool calling [...] allows autonomous systems to **complete** complex **tasks** by dynamically accessing and [may act] upon **external resources.**"

Real-life example

LLM as we've
known it so far

Find a bear near me!



LLM



Sorry, I don't know
which bears are near
you.

Real-life example

LLM as we've
known it so far

Find a bear near me!



LLM



Sorry, I don't know
which bears are near
you.

LLM with tools

<function API>
Find a bear near me!



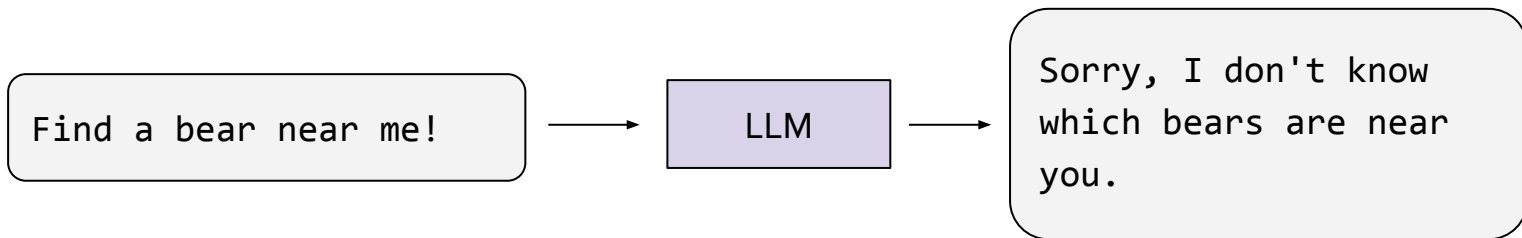
LLM



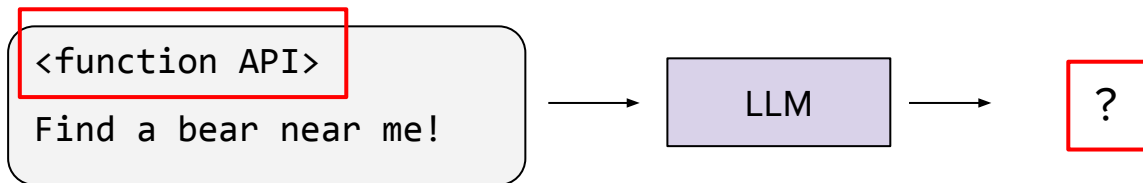
?

Real-life example

LLM as we've known it so far



LLM with tools



Real-life example

find_teddy_bear.py

```
from dataclasses import dataclass
from geopy.distance import geodesic
import requests

@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str

def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

    Parameters:
        location: A (latitude, longitude) pair representing the user's current
        location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    # Call API to get the closest teddy bear
    user_lat, user_lon = location
    api_url = "https://api.to.teddy.bears.com/v1/closest"
    try:
        response = requests.get(
            api_url,
            params={"latitude": user_lat, "longitude": user_lon},
            timeout=5
        )
        response.raise_for_status()
        closest_teddy_bear = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch teddy bear data from API: {e}")
```

```
# Extract coordinates from API response
bear_lat, bear_lon = closest_teddy_bear["coords"]

# Compute distance to the bear using geopy (returns distance in meters)
distance = geodesic((user_lat, user_lon), (bear_lat, bear_lon)).meters

return TeddyBearInfo(
    name=closest_teddy_bear["name"],
    distance_meters=round(distance, 2),
    mood=closest_teddy_bear["mood"],
    message=f"{closest_teddy_bear['name']} is {closest_teddy_bear['mood']} "
    f"and only {round(distance, 2)} meters away!"
)
```

Real-life example

find_teddy_bear.py

```
from dataclasses import dataclass
from geopy.distance import geodesic
import requests
```

```
@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str
```

```
def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

    Parameters:
        location: A (latitude, longitude) pair representing the user's current
        location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    # Call API to get the closest teddy bear
    user_lat, user_lon = location
    api_url = "https://api.to.teddy.bears.com/v1/closest"
    try:
        response = requests.get(
            api_url,
            params={"latitude": user_lat, "longitude": user_lon},
            timeout=5
        )
        response.raise_for_status()
        closest_teddy_bear = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch teddy bear data from API: {e}")
```

```
# Extract coordinates from API response
bear_lat, bear_lon = closest_teddy_bear["coords"]

# Compute distance to the bear using geopy (returns distance in meters)
distance = geodesic((user_lat, user_lon), (bear_lat, bear_lon)).meters

return TeddyBearInfo(
    name=closest_teddy_bear["name"],
    distance_meters=round(distance, 2),
    mood=closest_teddy_bear["mood"],
    message=f"{closest_teddy_bear['name']} is {closest_teddy_bear['mood']} "
    f"and only {round(distance, 2)} meters away!"
)
```

has a descriptive,
well-documented API

Real-life example

find_teddy_bear.py

```
from dataclasses import dataclass
from geopy.distance import geodesic
import requests

@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str

def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

    Parameters:
        location: A (latitude, longitude) pair representing the user's current
        location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    # Call API to get the closest teddy bear
    user_lat, user_lon = location
    api_url = "https://api.to.teddy.bears.com/v1/closest"
    try:
        response = requests.get(
            api_url,
            params={"latitude": user_lat, "longitude": user_lon},
            timeout=5
        )
        response.raise_for_status()
        closest_teddy_bear = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch teddy bear data from API: {e}")
```

```
# Extract coordinates from API response
bear_lat, bear_lon = closest_teddy_bear["coords"]

# Compute distance to the bear using geopy (returns distance in meters)
distance = geodesic((user_lat, user_lon), (bear_lat, bear_lon)).meters

return TeddyBearInfo(
    name=closest_teddy_bear["name"],
    distance_meters=round(distance, 2),
    mood=closest_teddy_bear["mood"],
    message=f"{closest_teddy_bear['name']} is {closest_teddy_bear['mood']} "
    f"and only {round(distance, 2)} meters away!"
)
```

(optional) has some
backend call

Real-life example

find_teddy_bear.py

```
from dataclasses import dataclass
from geopy.distance import geodesic
import requests
```

```
@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str
```

```
def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

    Parameters:
        location: A (latitude, longitude) pair representing the user's current
        location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    # Call API to get the closest teddy bear
    user_lat, user_lon = location
    api_url = "https://api.to.teddy.bears.com/v1/closest"
    try:
        response = requests.get(
            api_url,
            params={"latitude": user_lat, "longitude": user_lon},
            timeout=5
        )
        response.raise_for_status()
        closest_teddy_bear = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch teddy bear data from API: {e}")
```

```
# Extract coordinates from API response
bear_lat, bear_lon = closest_teddy_bear["coords"]

# Compute distance to the bear using geopy (returns distance in meters)
distance = geodesic((user_lat, user_lon), (bear_lat, bear_lon)).meters
```

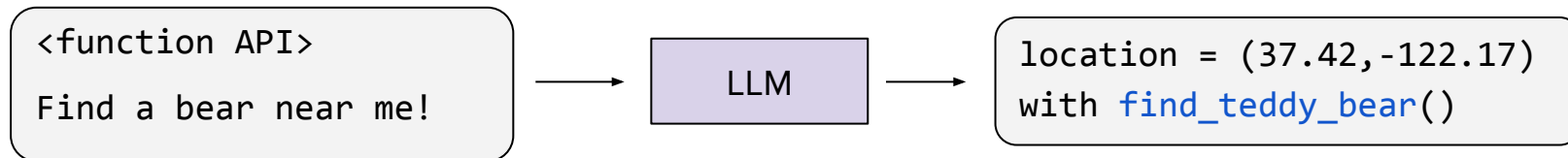
```
return TeddyBearInfo(
    name=closest_teddy_bear["name"],
    distance_meters=round(distance, 2),
    mood=closest_teddy_bear["mood"],
    message=f"{closest_teddy_bear['name']} is {closest_teddy_bear['mood']} "
    f"and only {round(distance, 2)} meters away!"
)
```

returns some info

How it works

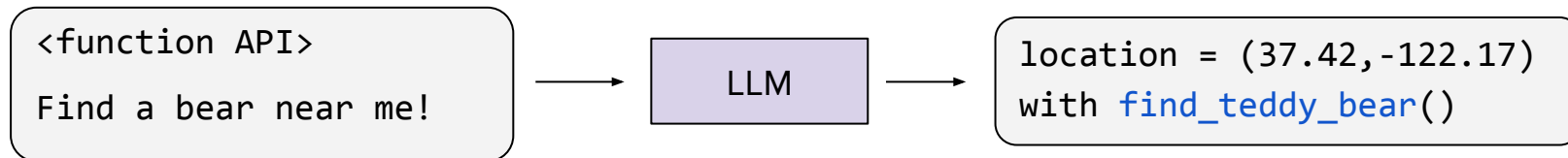
How it works

1. Let **LLM** find **argument** for **relevant function** call

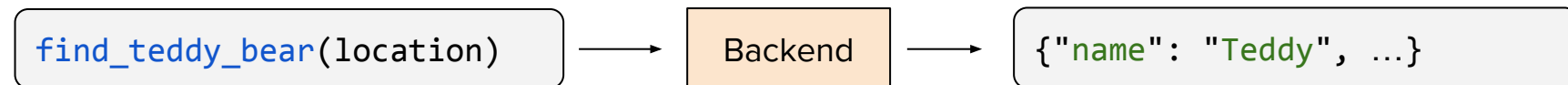


How it works

1. Let **LLM** find **argument** for **relevant function** call

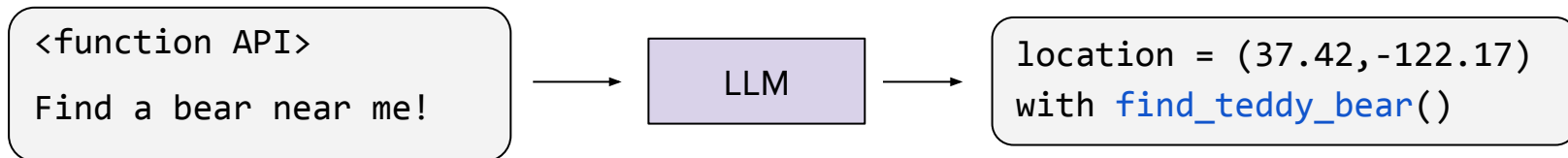


2. Make **function call**

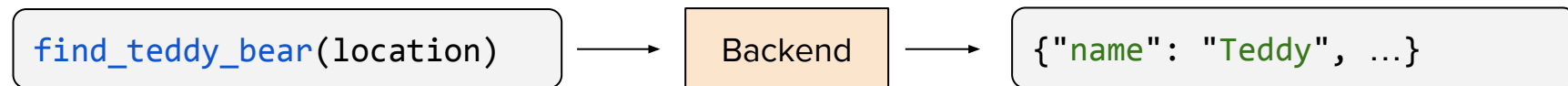


How it works

1. Let **LLM** find **argument** for **relevant function** call



2. Make **function call**



3. Let **LLM** deduce conclusion **based on results**



Teach a model to use a tool

Method 1: via training

Teach a model to use a tool

Method 1: via training

```
<function API>  
Find a bear near me!
```

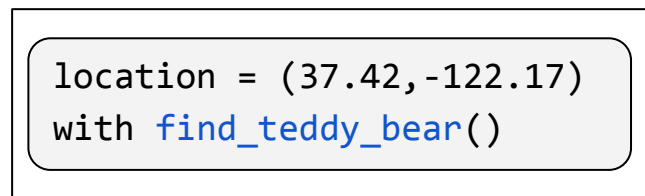
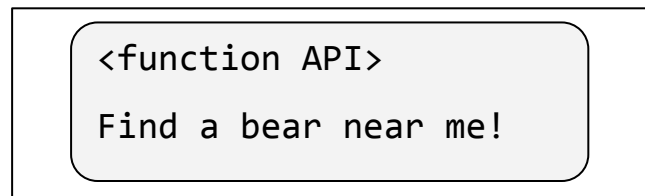


```
location = (37.42, -122.17)  
with find_teddy_bear()
```

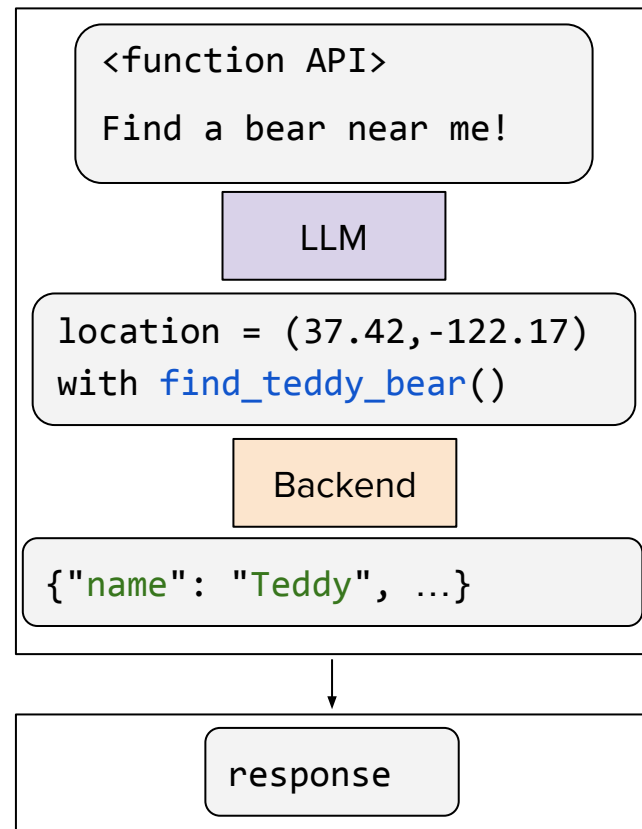
Tool prediction

Teach a model to use a tool

Method 1: via training



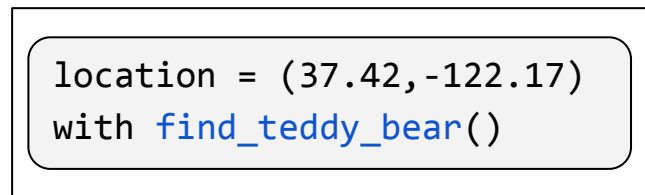
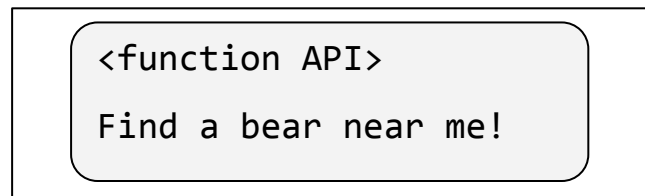
Tool prediction



Response generation

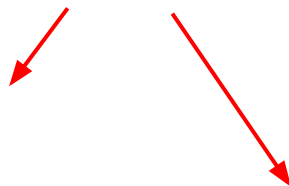
Teach a model to use a tool

Method 1: via training

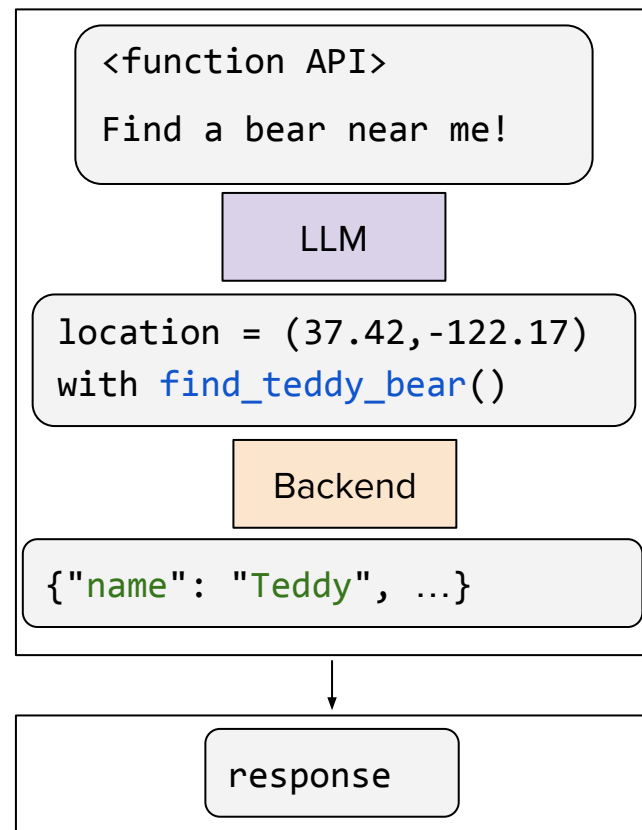


Tool prediction

conversation
history so far



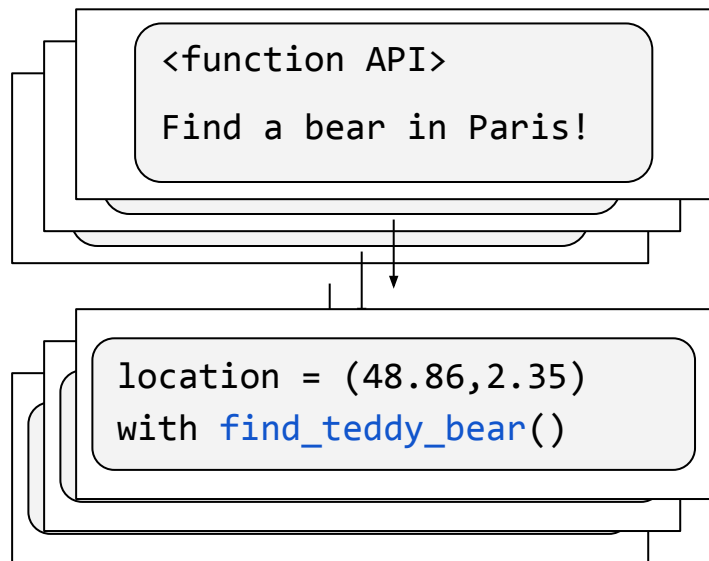
desired
prediction



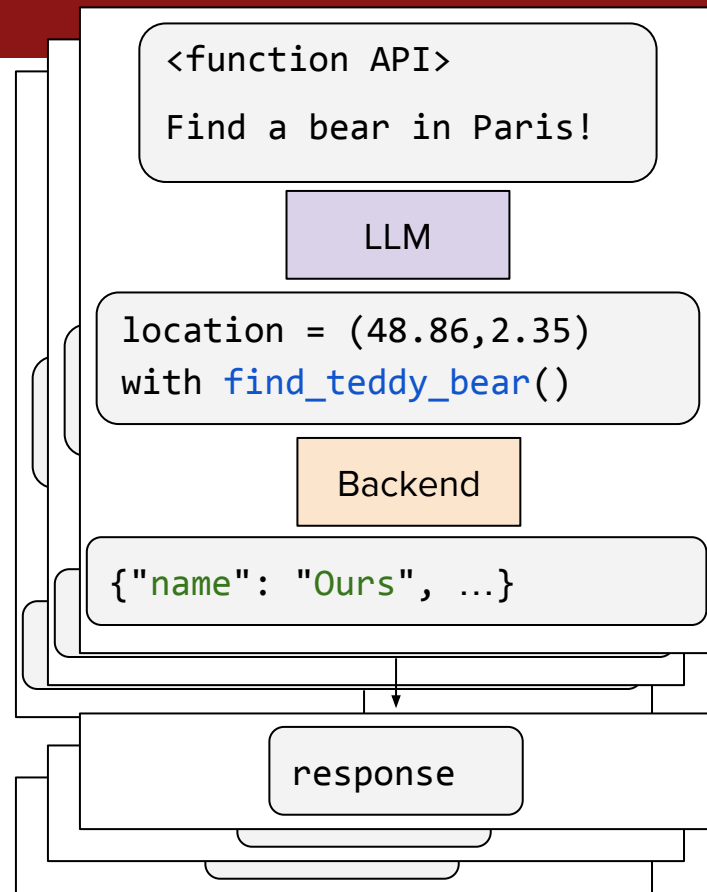
Response generation

Teach a model to use a tool

Method 1: via training



Tool prediction



Response generation

Teach a model to use a tool

Method 1: via training

Method 2: via prompting

`<function API> + <detailed explanation on how to use it>`

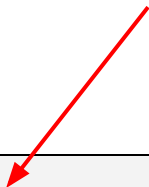
`Find a bear near me!`

Teach a model to use a tool

Method 1: via training

Method 2: via prompting

how to write such a
description?



`<function API> + <detailed explanation on how to use it>`

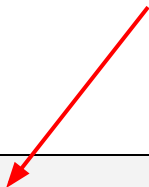
`Find a bear near me!`

Teach a model to use a tool

Method 1: via training

Method 2: via prompting

how to write such a
description?



```
<function API> + <detailed explanation on how to use it>
```

```
Find a bear near me!
```

One way:

Use SFT pairs as evaluation + use a powerful reasoning model to write it for you!

Examples of common use cases

Information

- Web/database search
- Weather, stocks, and any other tracker
- Codebase

Computation

- Calculator
- Code execution (often in Python)

...and many more!

Action

- Send emails/messages and other in-computer action
- anything else within the domain of an assistant

In practice: many tools

```
def find_teddy_bear(  
    location: tuple[float,  
                    float]  
) -> TeddyBearInfo:  
    # ...
```

```
def hug_teddy_bear(  
    recipient: TeddyBear,  
    intensity: str = "warm"  
) -> HugResponse:  
    # ...
```

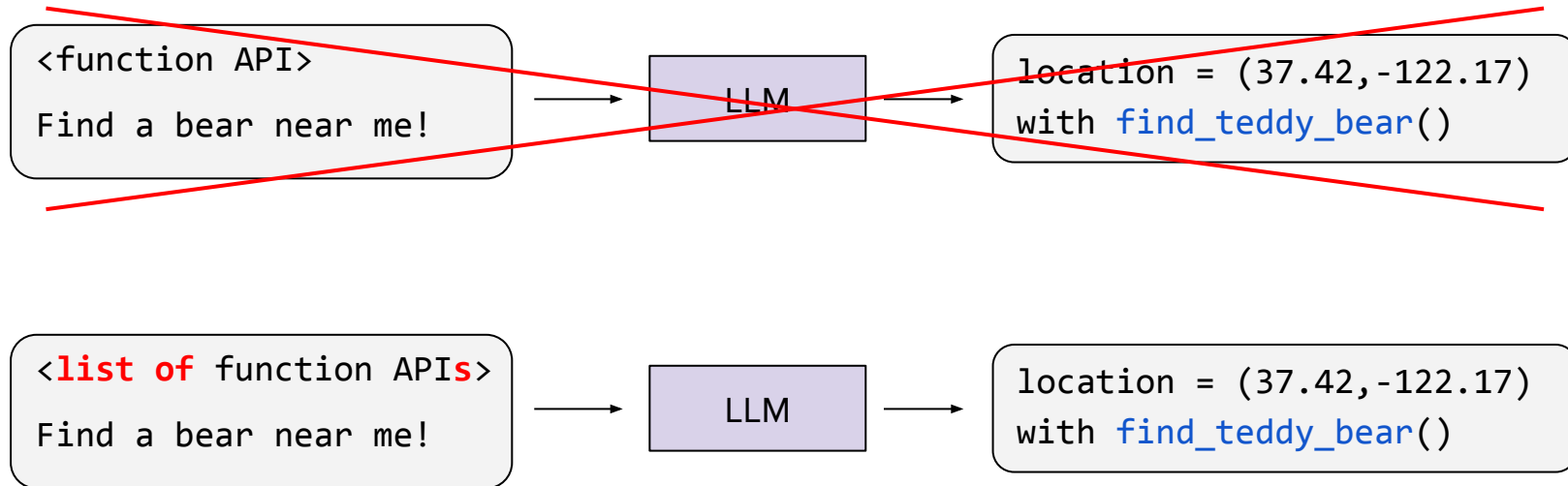
```
def check_teddy_mood(  
    name: TeddyBear  
) -> TeddyMood:  
    # ...
```

```
def send_teddy_gift(  
    recipient: TeddyBear,  
    gift: str = "poetry book"  
) -> GiftResponse:  
    # ...
```

```
def schedule_playdate(  
    host: TeddyBear,  
    guest: TeddyBear,  
    time: datetime  
) -> Confirmation:  
    # ...
```

```
def send_message(  
    recipient: TeddyBear,  
    message: str  
) -> MessageResponse:  
    # ...
```

In practice: many tools



Tools summary

Benefits.

- LLMs just became way more useful!
- They can also interact with the real world
- Overcomes "knowledge cutoff" limitation

Tools summary

Benefits.

- LLMs just became way more useful!
- They can also interact with the real world
- Overcomes "knowledge cutoff" limitation

Challenges.

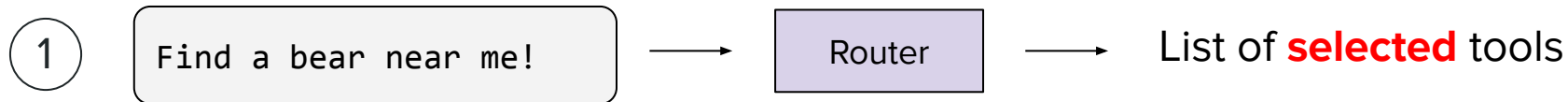
- More tools = decrease performance
- Finite context length: not scalable
- Many tools to define. Lots of work.

Tool selection

Goal. Both reduce latency and improve performance

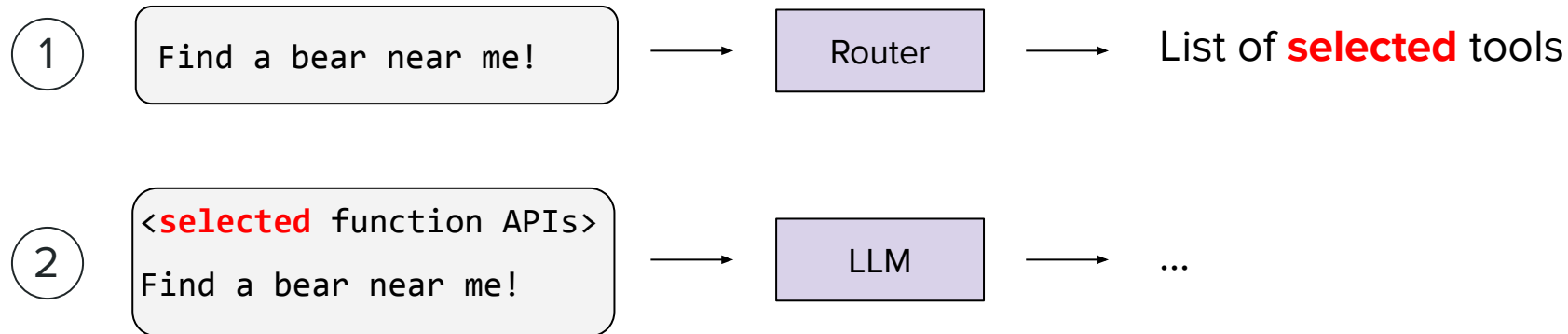
Tool selection

Goal. Both reduce latency and improve performance



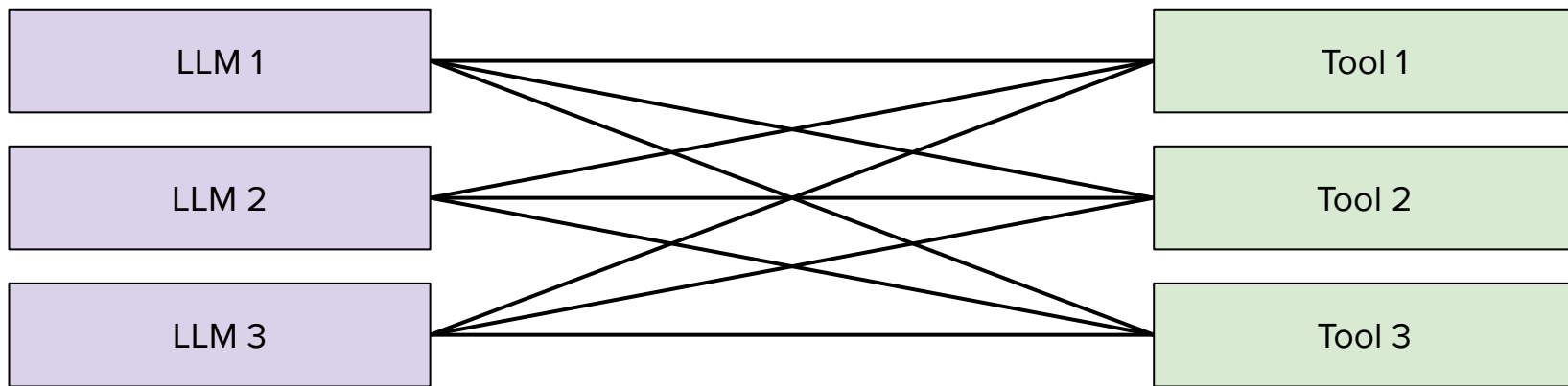
Tool selection

Goal. Both reduce latency and improve performance



Motivation for standardized protocol

Goal. Avoid duplication of tool implementations



Standardization: MCP

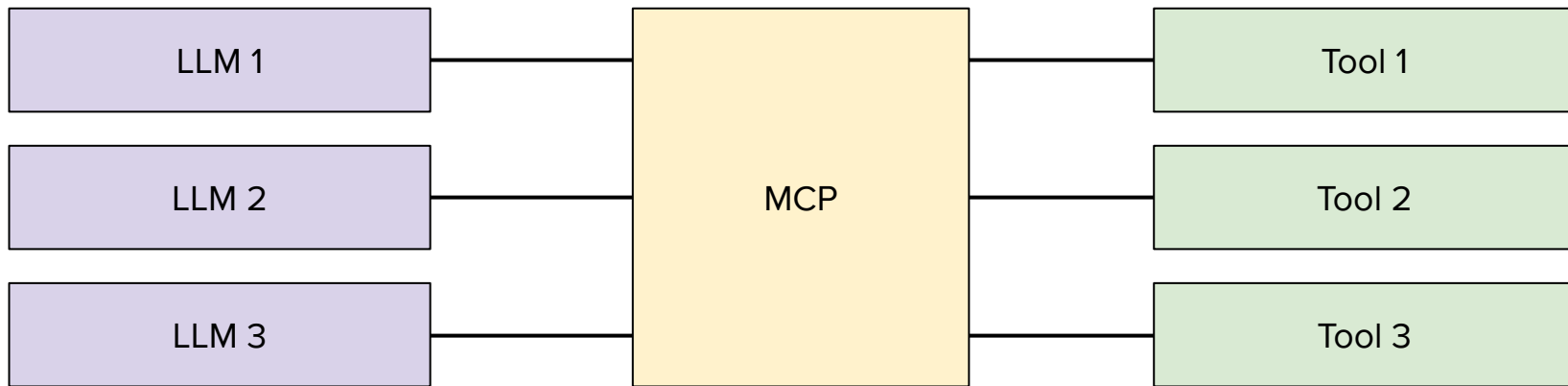
MCP = **M**odel **C**ontext **P**rotocol

Idea. Connect tools/data to LLMs in a standard way

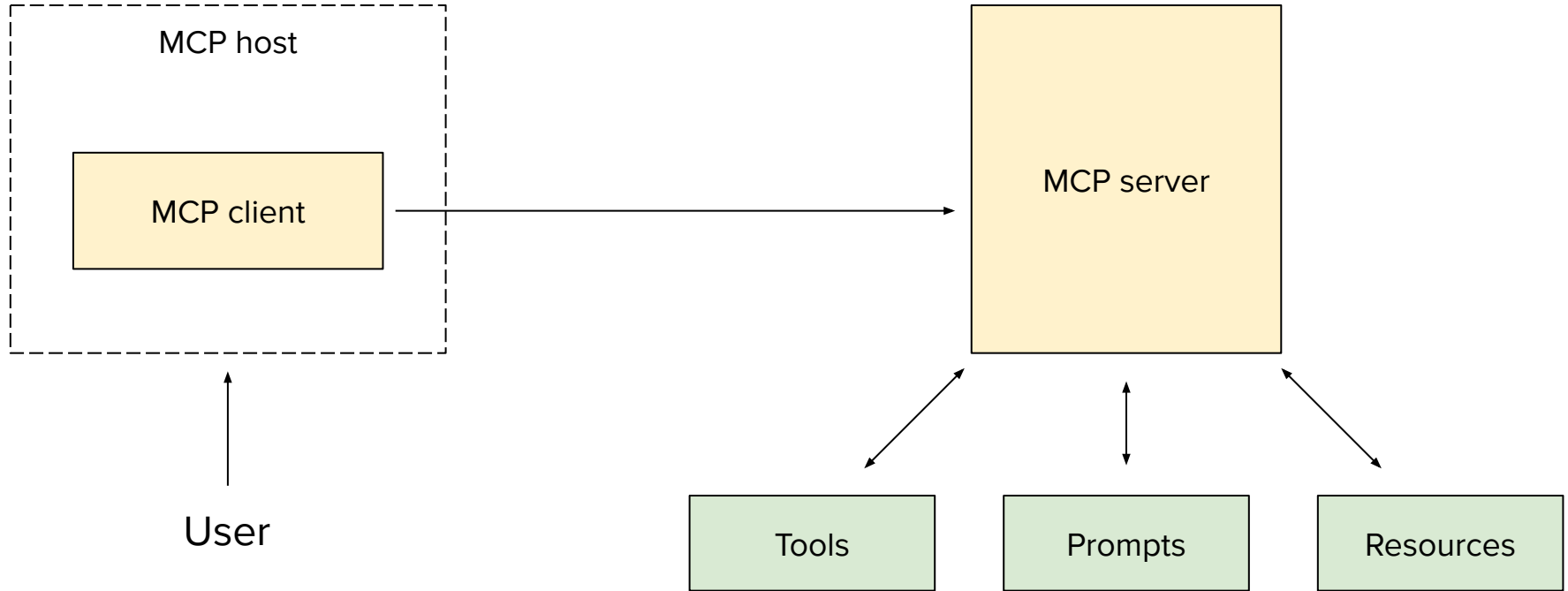
Standardization: MCP

MCP = **M**odel **C**ontext **P**rotocol

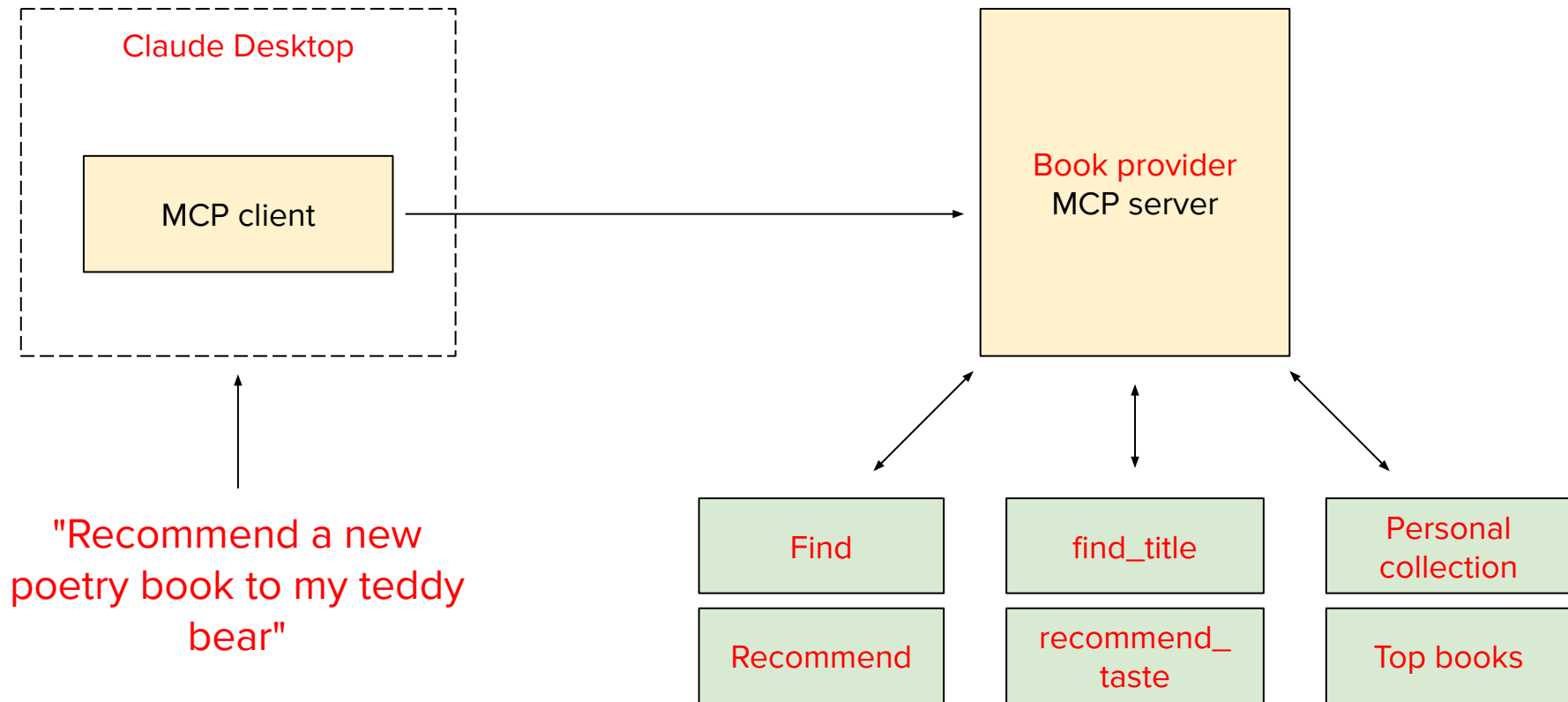
Idea. Connect tools/data to LLMs in a standard way



Standardization: MCP



Standardization: MCP





Transformers & Large Language Models

RAG

Tool calling

Agents

"An **agent** is a system that autonomously pursues goals and completes tasks on a user's behalf."

Definition

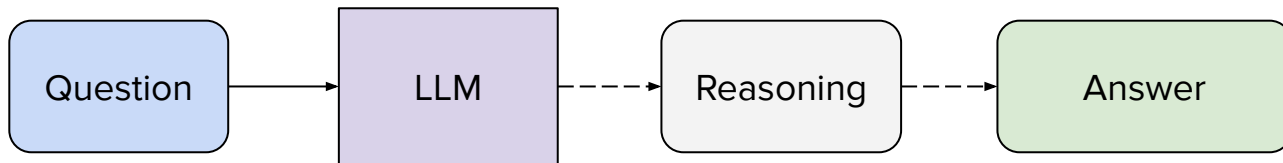
"An **agent** is a system that **autonomously**
pursues goals and **completes tasks** on a
user's behalf."

High-level idea

Traditional

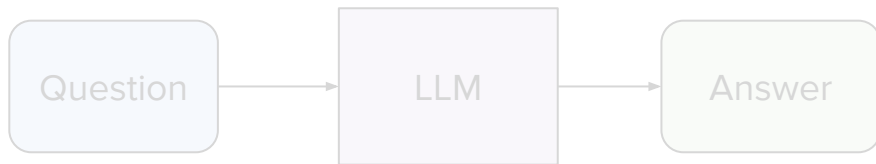


Reasoning

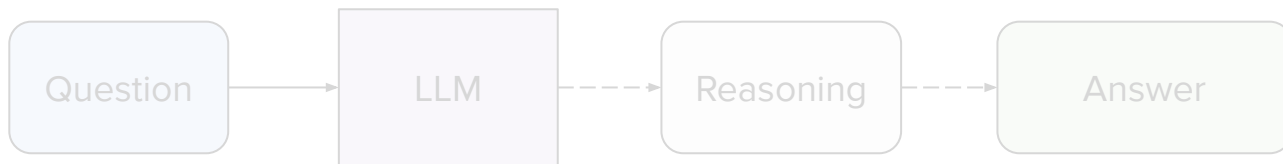


High-level idea

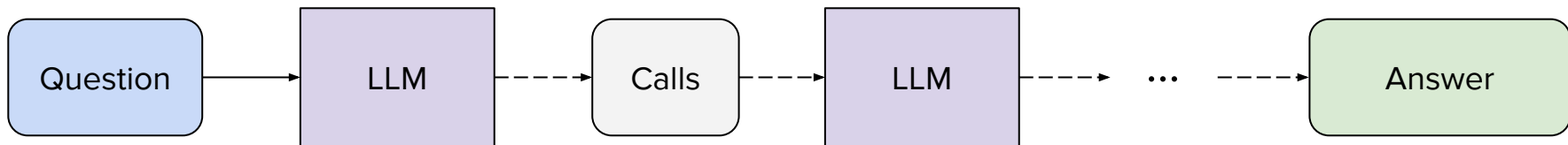
Traditional



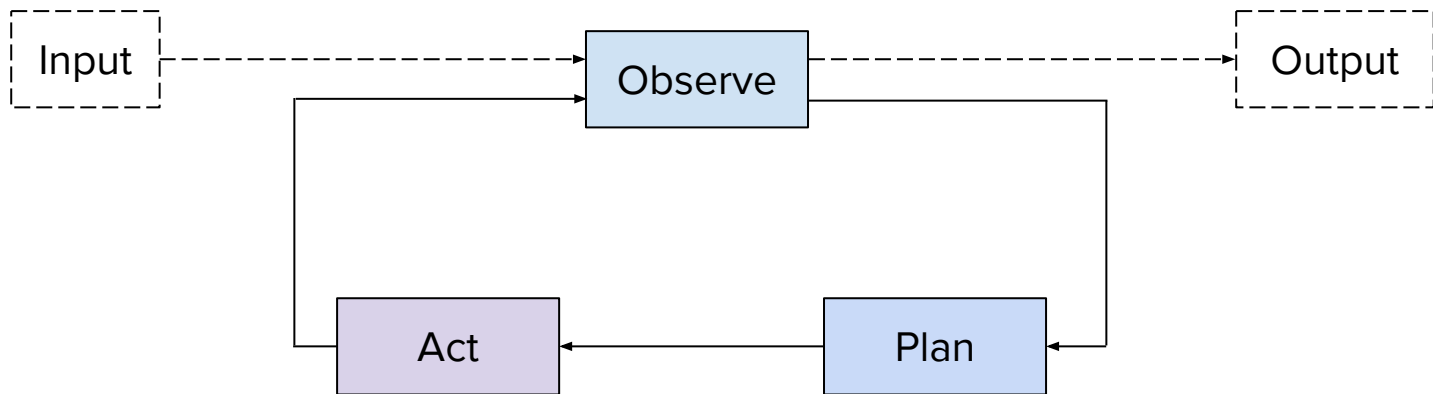
Reasoning



Agent



ReAct = Reason + **Act**



ReAct in action

Input

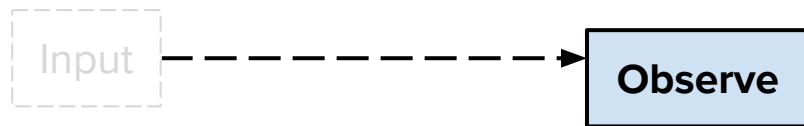
My teddy bear is cold.
Please do something.

Examples:

- Manually entered (e.g. user question)
- External event (e.g. metric going beyond a threshold)

ReAct in action

The user's teddy bear is cold, which may be due to the current temperature of the room, which is currently unknown.

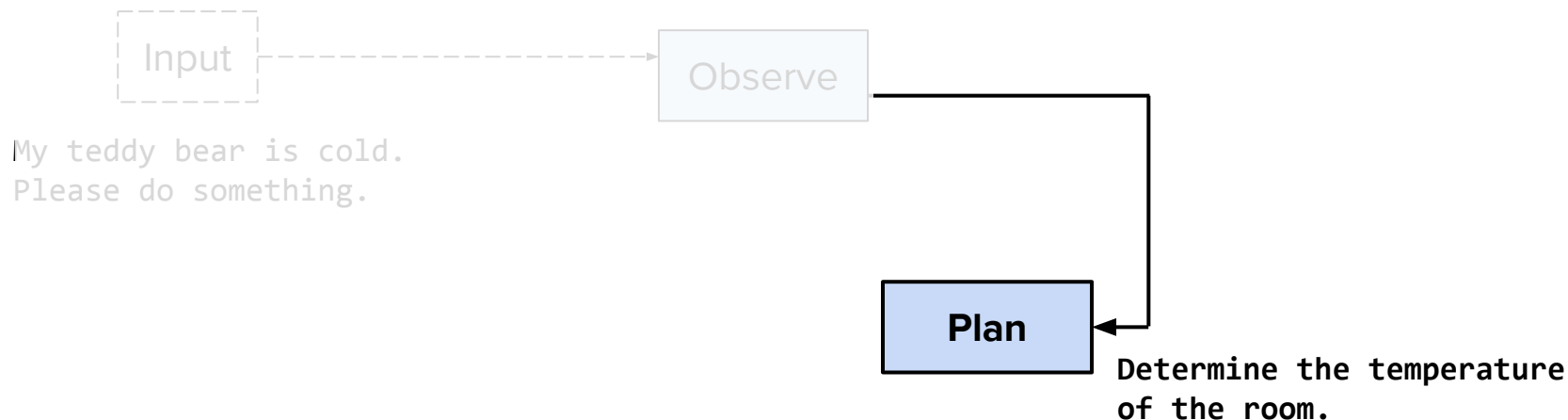


My teddy bear is cold.
Please do something.

- **Synthesize** previous actions + explicitly **state** what is **currently known** including own knowledge
- **Reasoning**-heavy step to figure out **what is needed**

ReAct in action

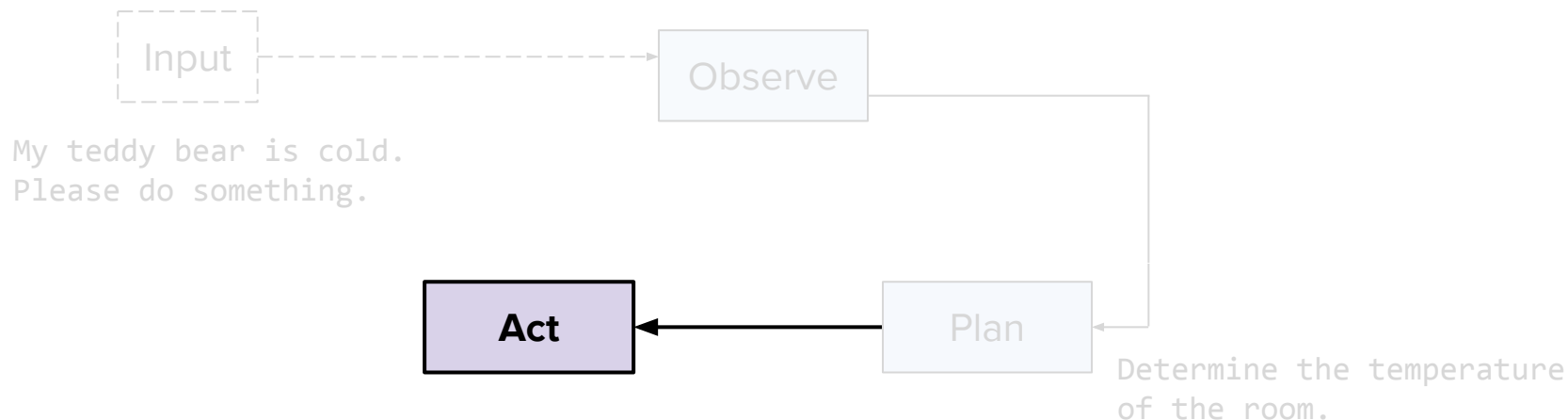
The user's teddy bear is cold, which may be due to the current temperature of the room, which is currently unknown.



Detail **what tasks** need to be accomplished and **what tools** to call

ReAct in action

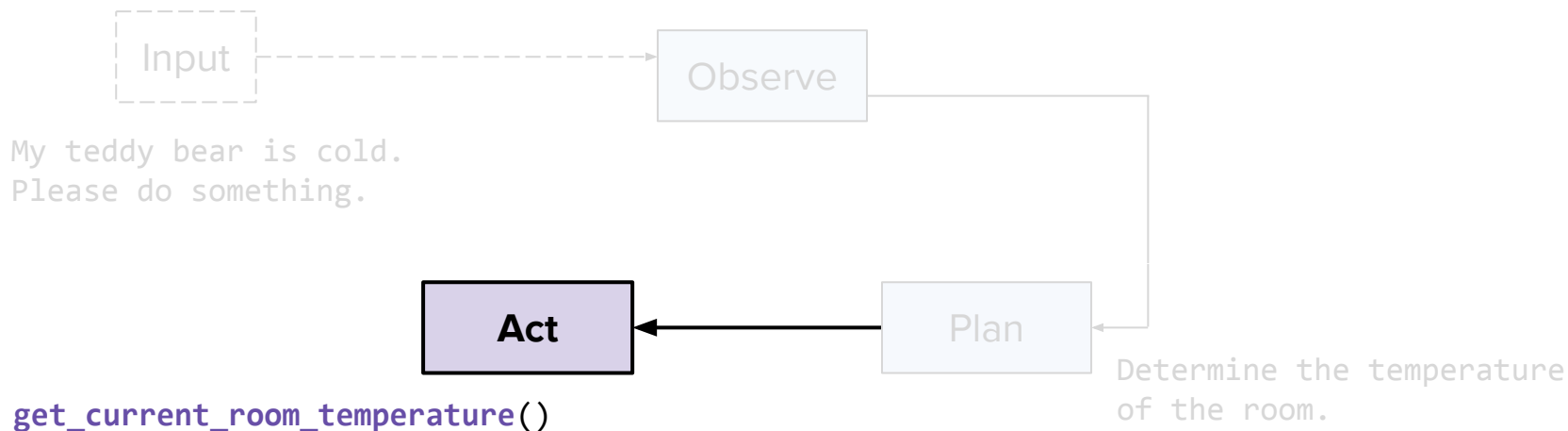
The user's teddy bear is cold, which may be due to the current temperature of the room, which is currently unknown.



- Perform an action via an **API**
- **Look** for info in a **database** of documents

ReAct in action

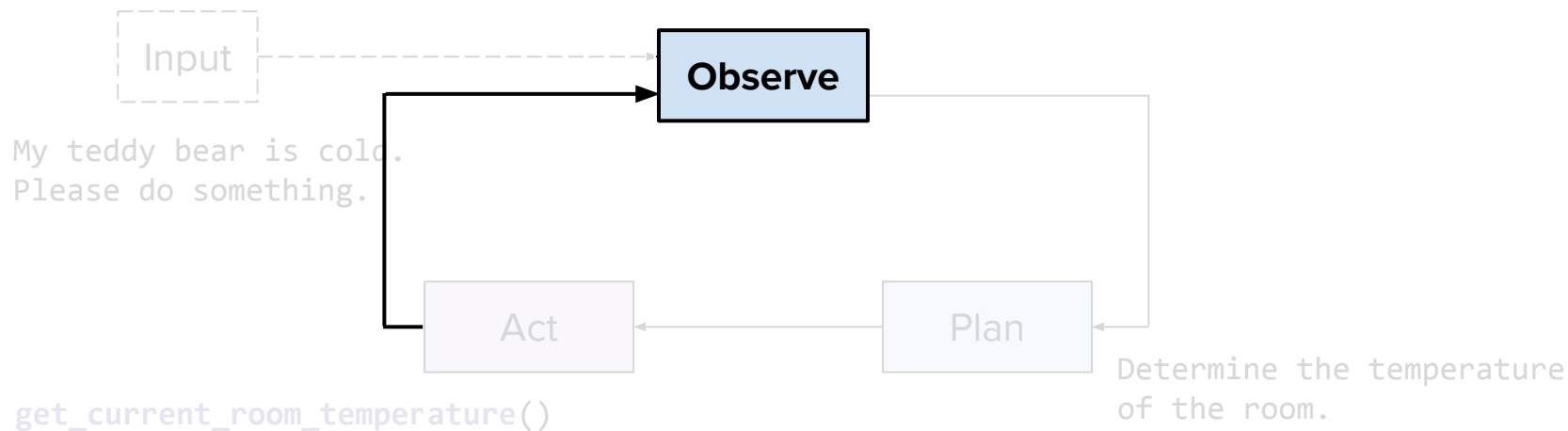
The user's teddy bear is cold, which may be due to the current temperature of the room, which is currently unknown.



- Perform an action via an **API**
- **Look** for info in a **database** of documents

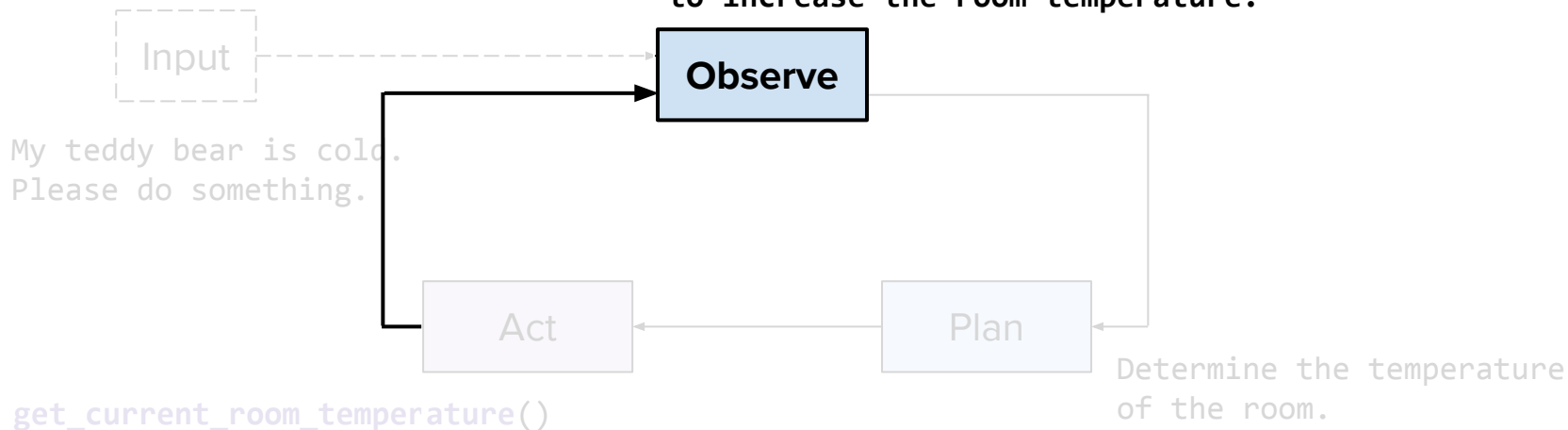
ReAct in action

The user's teddy bear is cold, which may be due to the current temperature of the room, which is currently unknown.



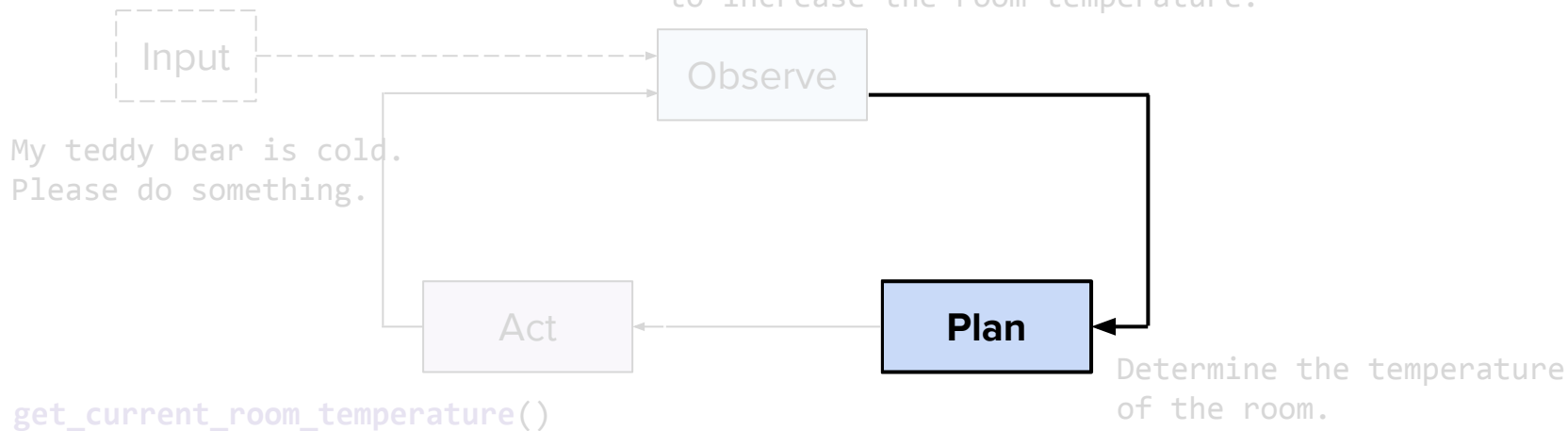
ReAct in action

The temperature in the room is currently 65F. This is about 5F less than an average temperature. We need to increase the room temperature.



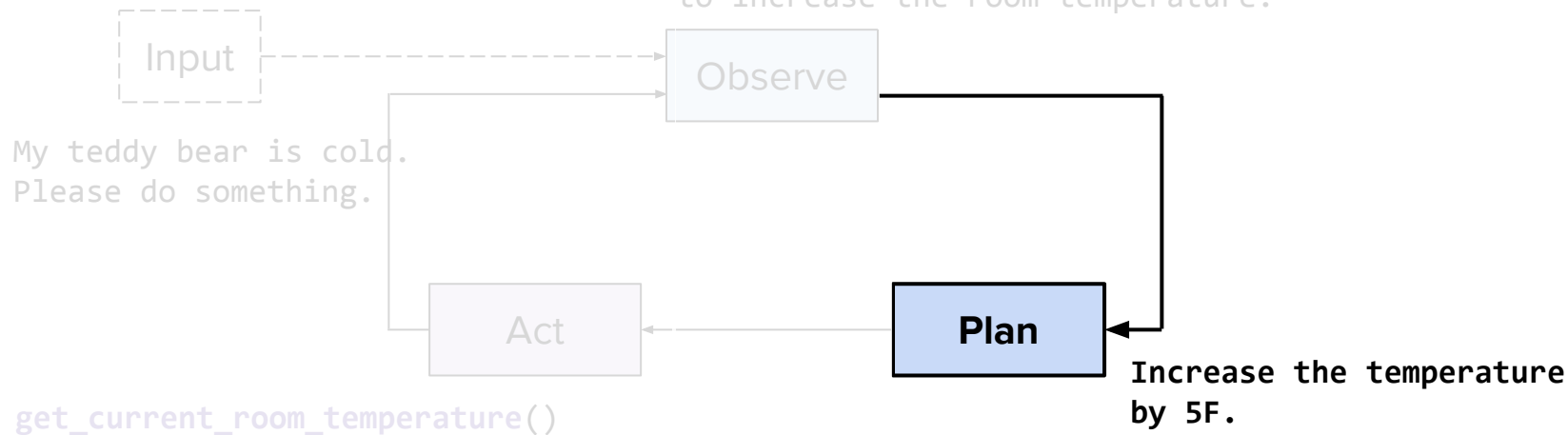
ReAct in action

The temperature in the room is currently 65F. This is about 5F less than an average temperature. We need to increase the room temperature.



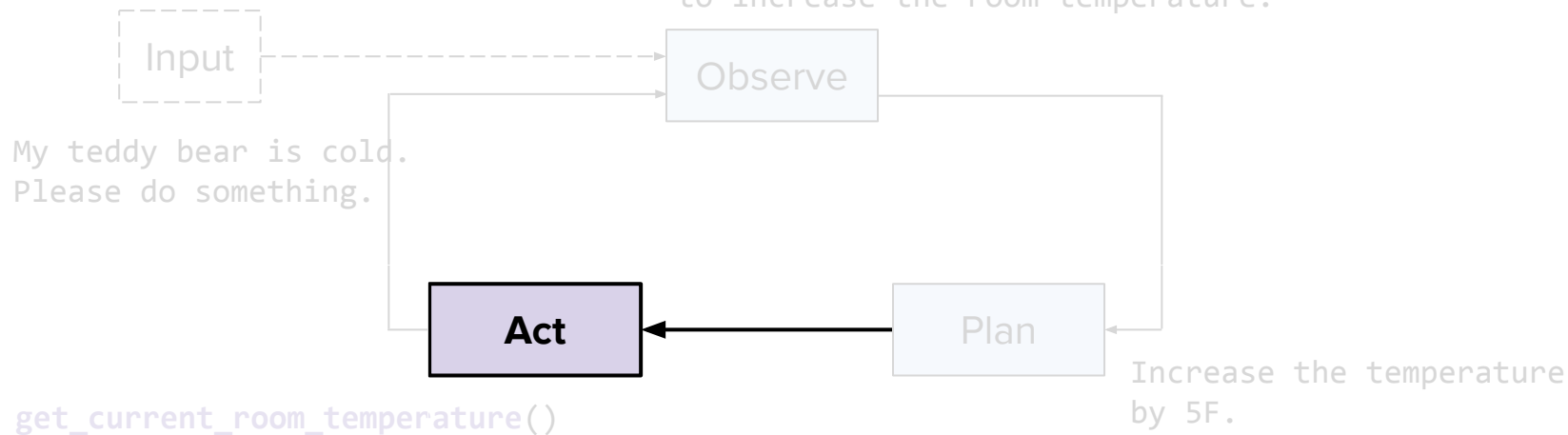
ReAct in action

The temperature in the room is currently 65F. This is about 5F less than an average temperature. We need to increase the room temperature.



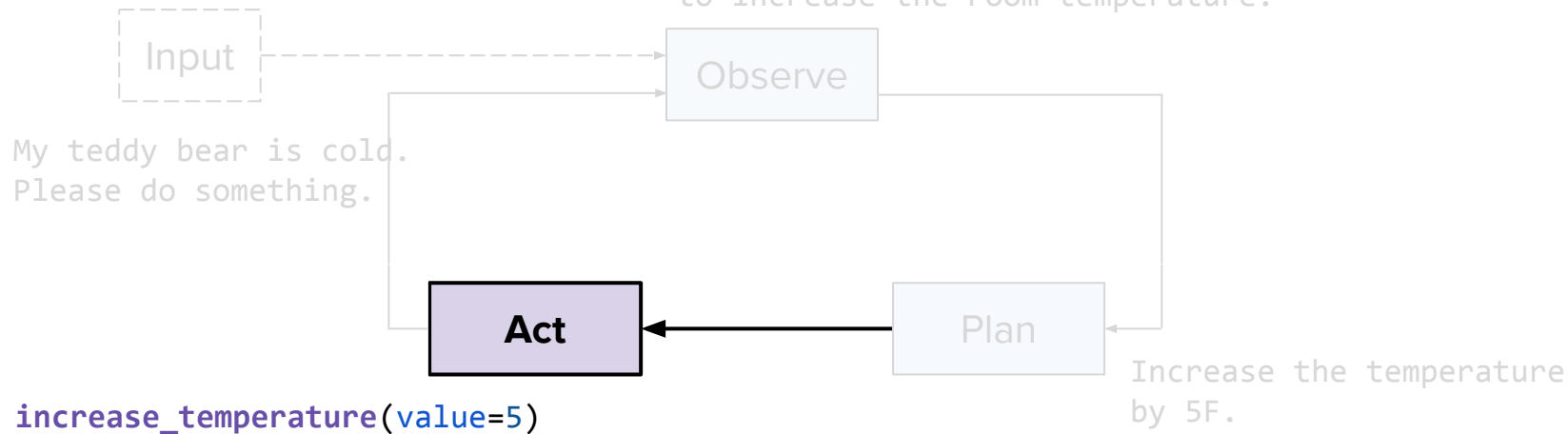
ReAct in action

The temperature in the room is currently 65F. This is about 5F less than an average temperature. We need to increase the room temperature.



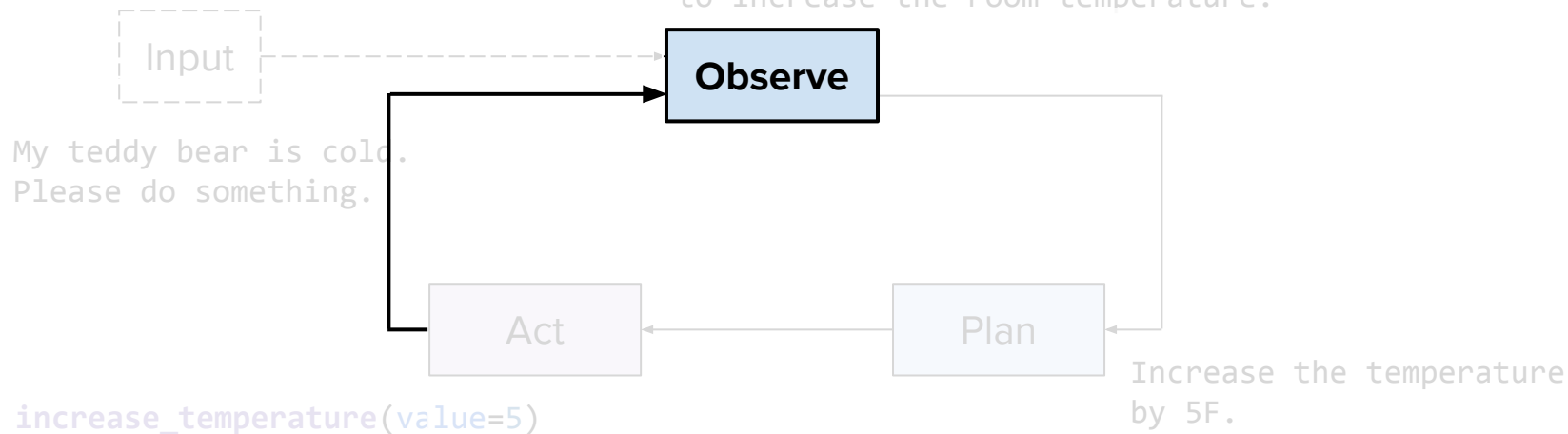
ReAct in action

The temperature in the room is currently 65F. This is about 5F less than an average temperature. We need to increase the room temperature.



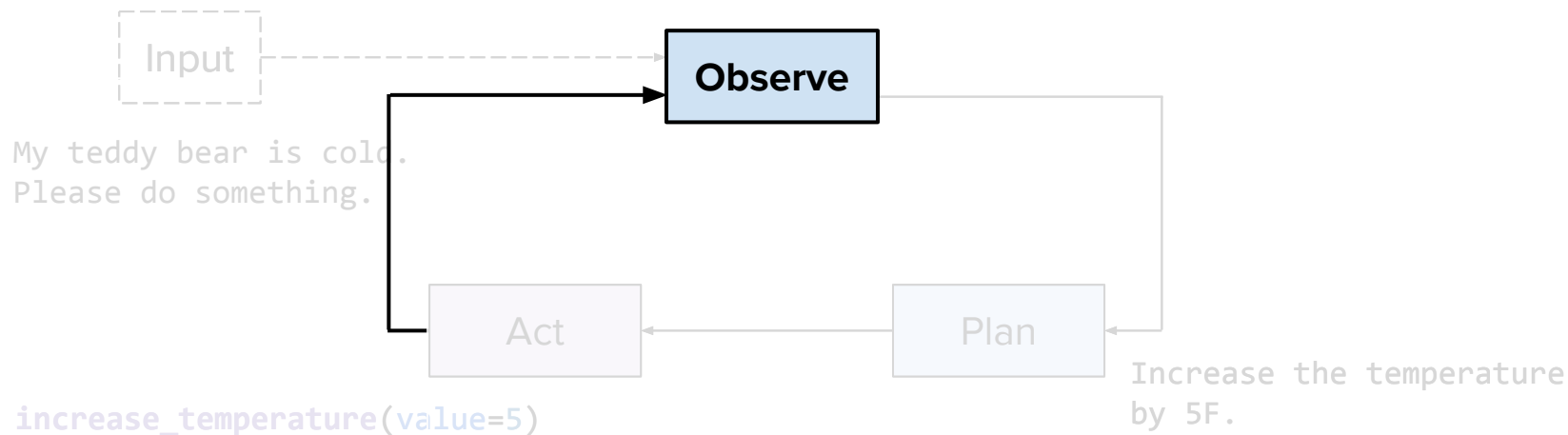
ReAct in action

The temperature in the room is currently 65F. This is about 5F less than an average temperature. We need to increase the room temperature.



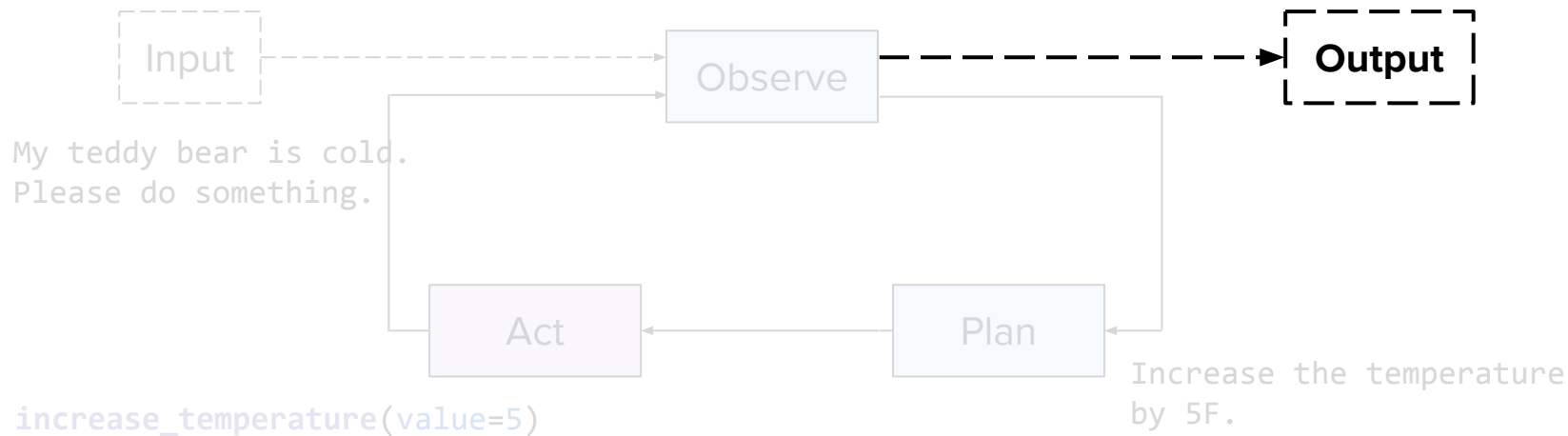
ReAct in action

The thermostat is now set to 70F. This should be warm enough.



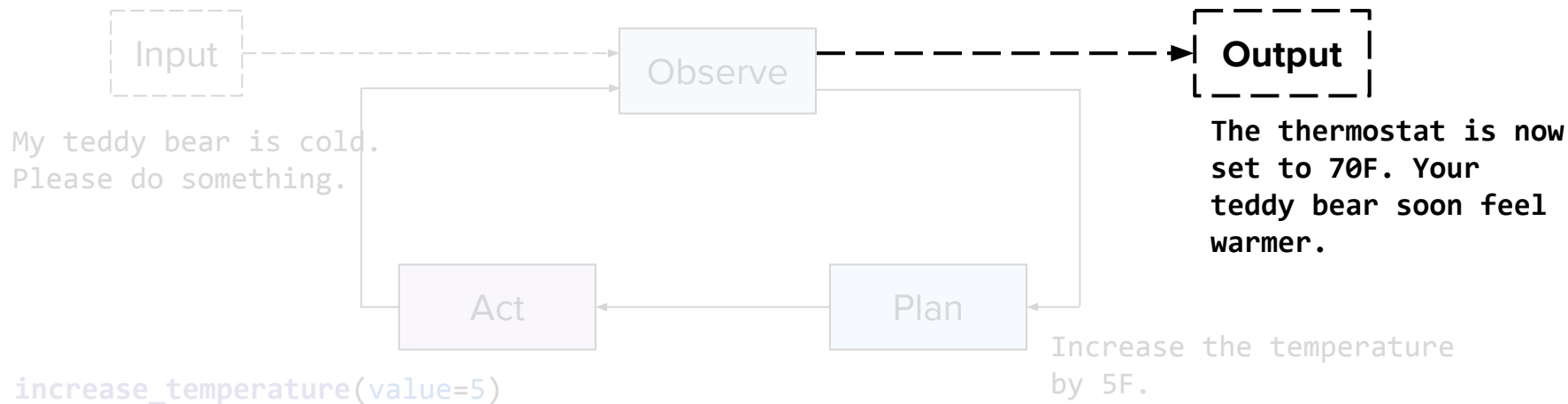
ReAct in action

The thermostat is now set to 70F. This should be warm enough.

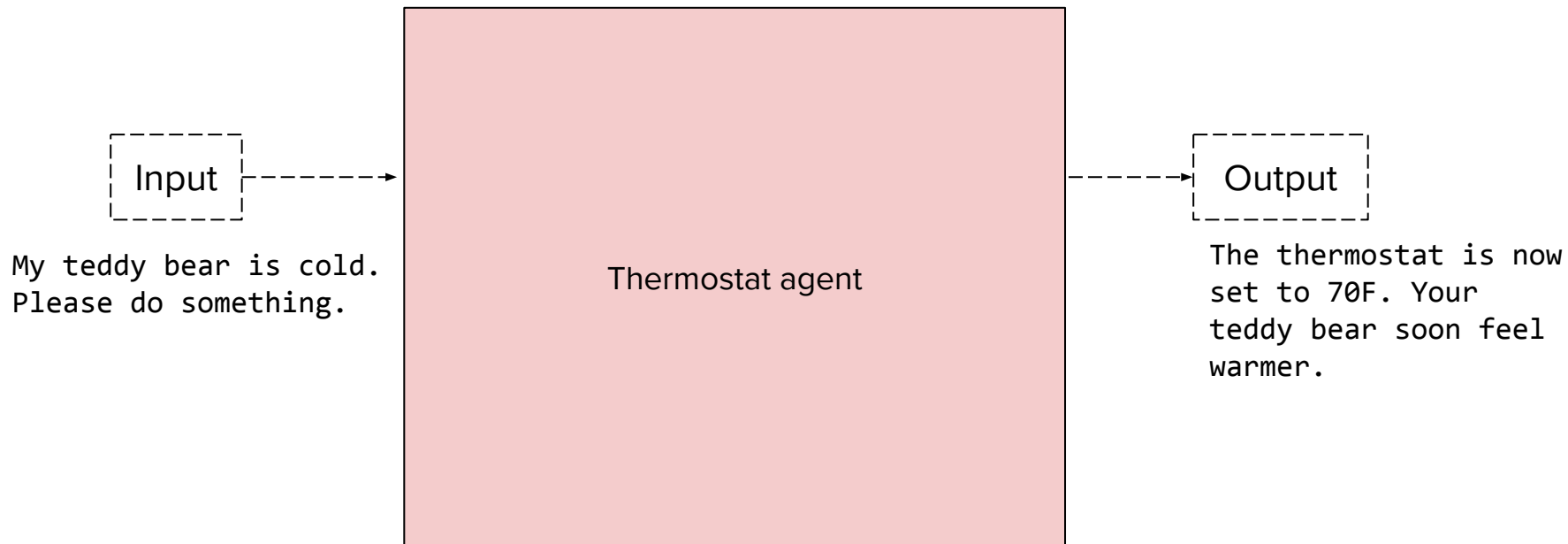


ReAct in action

The thermostat is now set to 70F. This should be warm enough.



"Agentic" view



There can be agents for many things

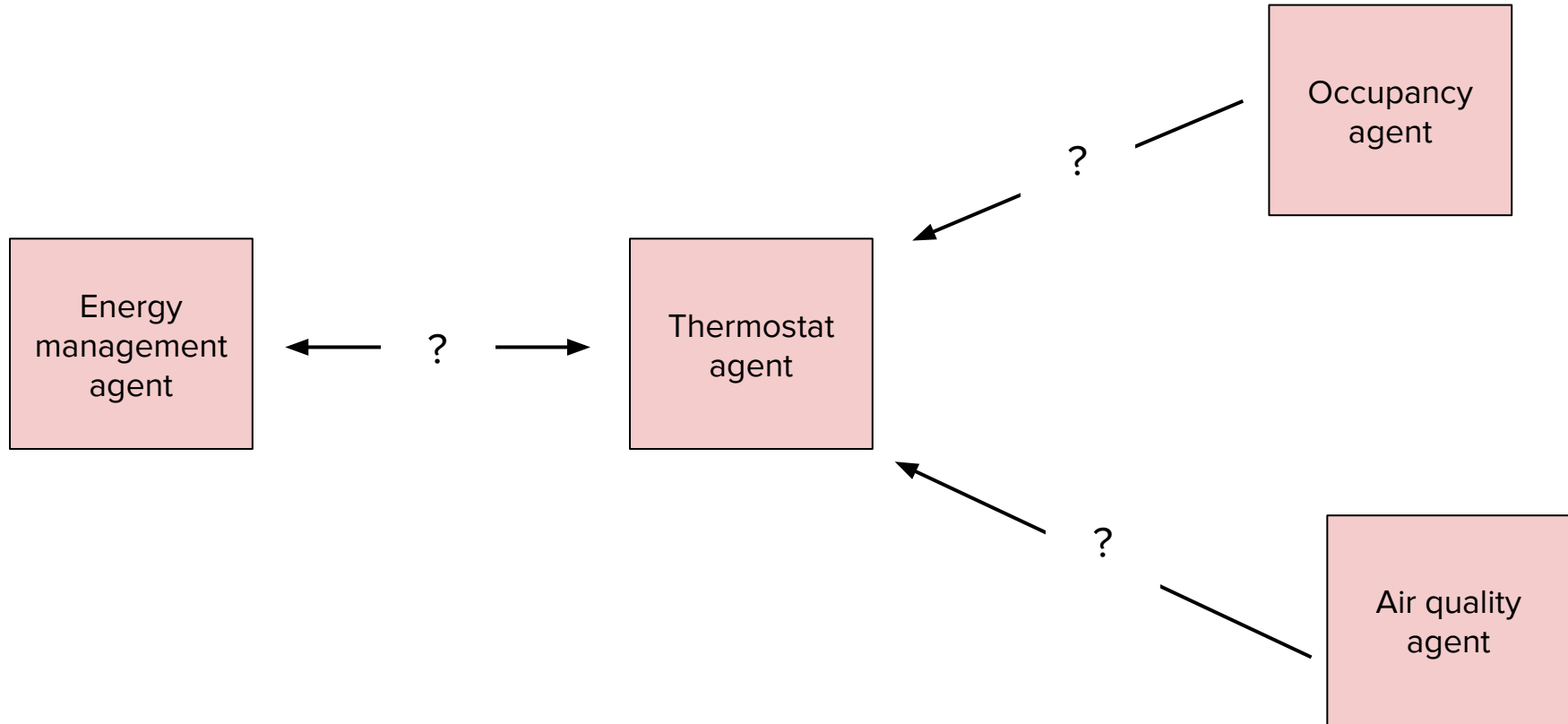
Energy
management
agent

Thermostat
agent

Occupancy
agent

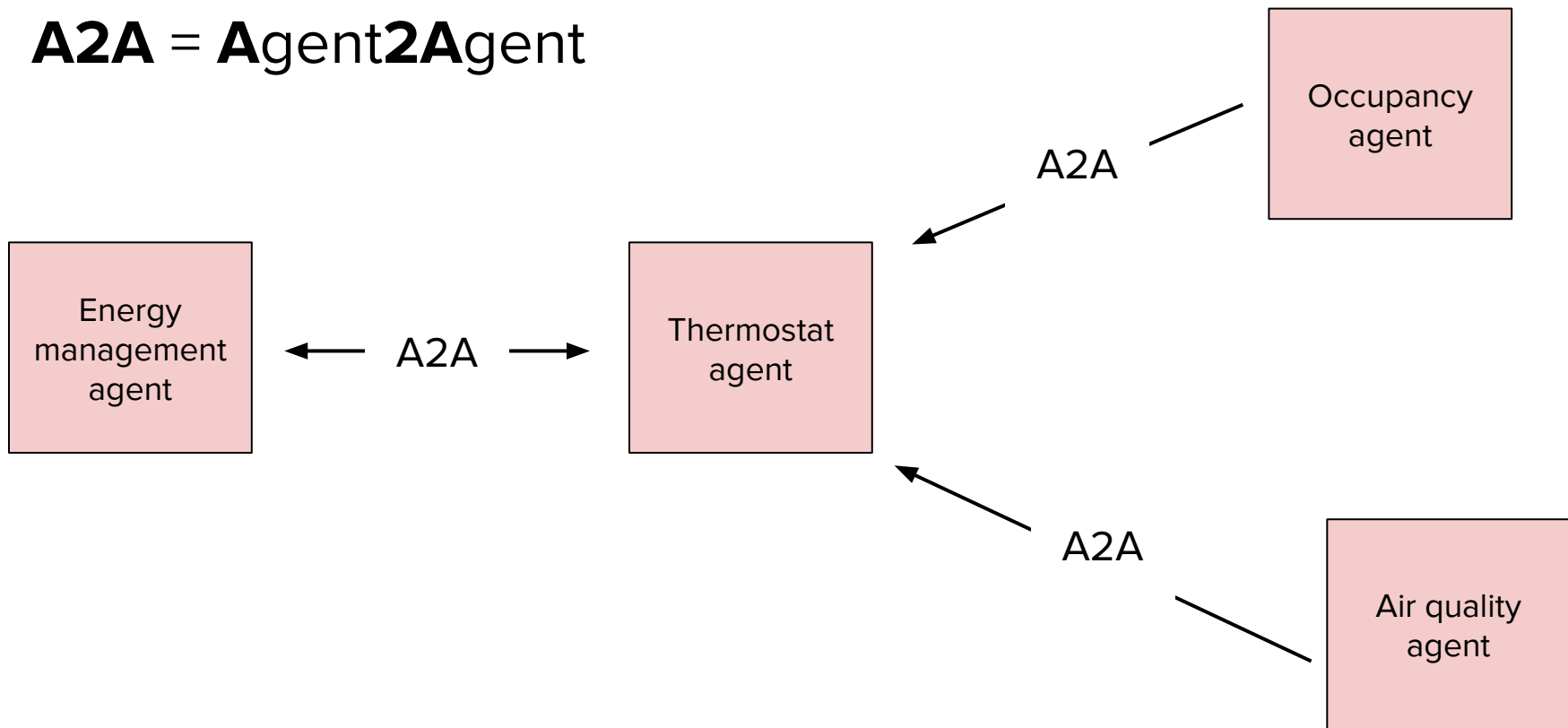
Air quality
agent

How can agents communicate with each other?

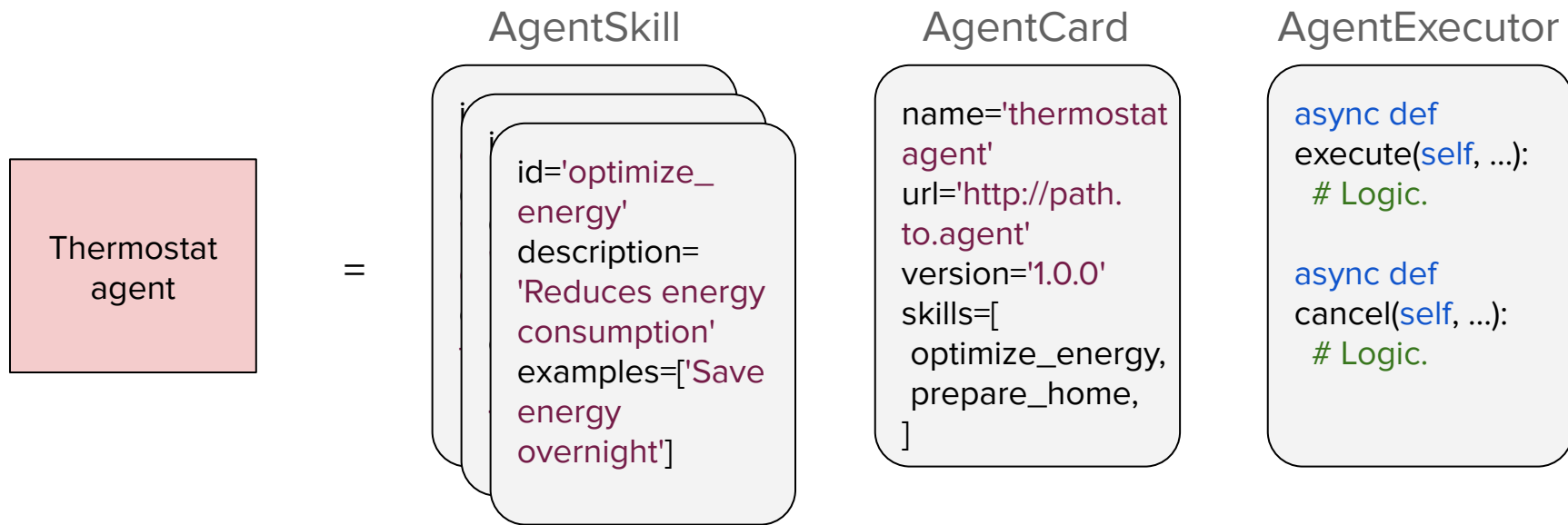


Standardization: A2A

A2A = Agent2Agent



Standardization: A2A



Safety

Risks.

- Potential for harm in the real world
- Example: data exfiltration

Safety

Risks.

- Potential for harm in the real world
- Example: data exfiltration

Remediations.

- Training steps
- Inference safeguards
- Benchmarks, e.g. **Agent-SafetyBench**

Safety

Risks.

- Potential for harm in the real world
- Example: data exfiltration

Remediations.

- Training steps
- Inference safeguards
- Benchmarks, e.g. [Agent-SafetyBench](#)

...very important topic!

Just yesterday in the news



Closing thoughts

- **Hallucination** is a (big) problem

Closing thoughts

- **Hallucination** is a (big) problem
- **Reasoning** abilities are a bottleneck
 - Finetuning helps, but hard
 - New capabilities are very welcome

Closing thoughts

- **Hallucination** is a (big) problem
- **Reasoning** abilities are a bottleneck
 - Finetuning helps, but hard
 - New capabilities are very welcome
- **Evaluation** is challenging

Closing thoughts

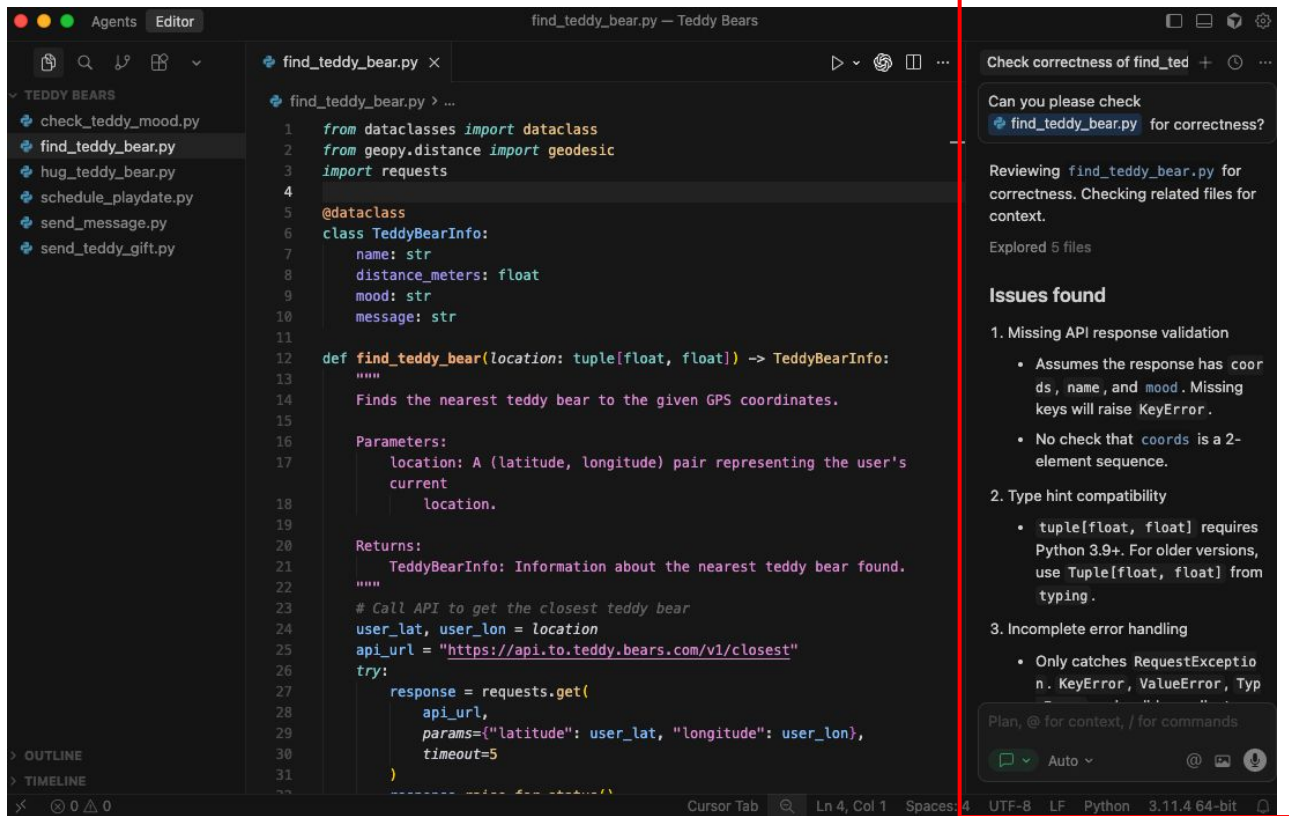
- **Hallucination** is a (big) problem
- **Reasoning** abilities are a bottleneck
 - Finetuning helps, but hard
 - New capabilities are very welcome
- **Evaluation** is challenging
- Good to **start simple**, then **iterate** and **progressively scale up**
- Good to **start** with **capable models**, **optimize** on **size later**

Closing thoughts

- **Hallucination** is a (big) problem
- **Reasoning** abilities are a bottleneck
 - Finetuning helps, but hard
 - New capabilities are very welcome
- **Evaluation** is challenging
- Good to **start simple**, then **iterate** and **progressively scale up**
- Good to **start** with **capable models**, **optimize** on **size later**
- **Transparency** / **observability** helps with user trust and debuggability

Bonus: AI agents in your daily life

Personal favorite use case: coding!



The image shows a code editor window titled "find_teddy_bear.py — Teddy Bears" with a file explorer on the left listing several Python files. The main editor displays the code for `find_teddy_bear.py`, which includes imports for `dataclasses`, `geopy.distance`, and `requests`, a `TeddyBearInfo` dataclass, and a `find_teddy_bear` function that makes an API call to find the nearest teddy bear.

To the right of the code editor is a sidebar interface for an AI agent. A red arrow points from the word "agent" to this sidebar. The sidebar contains a chat window with the message: "Can you please check find_teddy_bear.py for correctness?". Below this, it states "Reviewing find_teddy_bear.py for correctness. Checking related files for context. Explored 5 files". Under the heading "Issues found", there are three items:

1. Missing API response validation
 - Assumes the response has `coords`, `name`, and `mood`. Missing keys will raise `KeyError`.
 - No check that `coords` is a 2-element sequence.
2. Type hint compatibility
 - `tuple[float, float]` requires Python 3.9+. For older versions, use `Tuple[float, float]` from typing.
3. Incomplete error handling
 - Only catches `RequestException`. `KeyError`, `ValueError`, `TypeError`, etc.

At the bottom of the sidebar, there is a prompt area with the text "Plan, @ for context, / for commands" and a button labeled "Auto".

Thank you for your attention!
